

claude-windows / 96963dab-d38f-488c-af44-530ddbfeed1

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbfeed1\subagents\workflows\wf_00bebd3c-82f\agent-a0f54d88b459b2227.jsonl`
- SHA-256: `053154c9420a54380fbce084703bdcc6436cd7ab1aa26f318ef1bfaaedbfc530`
- Source modified: `2026-07-08T05:51:07+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_00bebd3c-82f`
- session_id: `96963dab-d38f-488c-af44-530ddbfeed1`
```

Transcript

1. user (2026-07-08T05:44:49.639Z)

You are mapping a subsystem of the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (Windows checkout; read-only ? do NOT edit anything). Context: GitHub issue #524 wants to (a) move validation onto the L4 Cloud Run Job worker via a config-driven phase_placement knob, (b) drop a redundant re-hash in /api/process by trusting the upload's streamed video_id, and (c) design a topology where upload->stitch runs on the L4 with async copy-to-bucket. You are producing the precise code map a designer/implementer needs. Cite exact file paths and line numbers. Read the actual code, not just docs. Follow imports where the flow crosses files.

Subsystem: STORAGE BACKENDS. Map backend/storage/ (base ABC + registry, local/gcs/gdrive/gdrive_api/s3): acquire_local_input semantics (download-to-scratch? streaming? caching?), how uploads/mirroring to the bucket work (is there an async copy primitive?), URI schemes, scratch dir lifecycle, and any existing co-location/region assumptions. Note which primitives exist vs would need building for an "async copy-to-bucket during segmentation" durability model.

2. user (2026-07-08T05:45:01.029Z)

```
backend\storage\gdrive.py
backend\storage\local.py
backend\storage\capabilities.py
backend\storage\gdrive_api.py
backend\storage\s3.py
backend\storage\ref.py
backend\storage\__init__.py
backend\storage\base.py
backend\storage\gcs.py
```

3. user (2026-07-08T05:45:03.637Z)

```
245 backend/storage/__init__.py
75 backend/storage/base.py
138 backend/storage/capabilities.py
157 backend/storage/gcs.py
148 backend/storage/gdrive.py
263 backend/storage/gdrive_api.py
72 backend/storage/local.py
111 backend/storage/ref.py
96 backend/storage/s3.py
1305 total
```

4. user (2026-07-08T05:45:13.433Z)

```
1      """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
```

```

3      Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
backends
4      live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5      pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted
Drive ?
6      no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7      """
8
9      from __future__ import annotations
10
11     import os
12     from abc import ABC, abstractmethod
13     from typing import IO, Iterator
14
15     from backend.storage.ref import StorageRef, StorageStat
16
17
18     class StorageBackend(ABC):
19         """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
20         worker dispatcher needs; the rest support listing/checkpointing/serving."""
21
22         @abstractmethod
23         def fetch_to_local(
24             self,
25             ref: StorageRef,
26             dst: "str | os.PathLike[str] | None" = None,
27             *,
28             overwrite: bool = False,
29         ) -> str:
30             """Make the object available as a local file; return its local path. For the
31             ``local`` backend this is an identity passthrough (the object is already
local)."""
32
33         @abstractmethod
34         def put(
35             self,
36             local_path: "str | os.PathLike[str]",
37             ref: StorageRef,
38             *,
39             content_type: "str | None" = None,
40         ) -> StorageRef:
41             """Upload/copy a local file to ``ref``; return ``ref``."""
42
43         @abstractmethod
44         def exists(self, ref: StorageRef) -> bool: ...
45
46         @abstractmethod
47         def stat(self, ref: StorageRef) -> StorageStat: ...
48
49         @abstractmethod
50         def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
51             """Yield a ``StorageRef`` per object under ``prefix``."""
52
53         @abstractmethod
54         def delete(self, ref: StorageRef) -> None: ...
55
56         def signed_url(
57             self, ref: StorageRef, *, ttl: "int | None" = None, method: str = "GET"
58         ) -> str:
59             """A time-limited URL handing the bytes to a browser/LLM. ``ttl=None`` (the
default)
60             means "use the backend's configured ``signed_url_ttl``" ? so operator config
wins
61             unless a caller explicitly needs a different expiry (#469). Not every backend

```

```

can
62         sign (`local` cannot; Drive only emulates) ? callers must treat
63         ``NotImplementedError`` as expected and fall back to ``fetch_to_local``.
64         raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
65
66     def open_writer(
67         self, ref: StorageRef, *, content_type: "str | None" = None
68     ) -> IO[bytes]:
69         """A writable binary file-like that streams bytes to ``ref`` AS THEY ARE
WRITTEN,
70         finalizing the object on ``close()``. Lets a server stage a large incoming
upload
71         without ever assembling it on local disk/tmpfs (CONTROL_PLANE_DURABLE_REBUILD P2
?
72         the 4 GiB RAM-``/tmp`` control plane OOM'd on a 2.68 GB browser upload, #480).
73         Backends that cannot stream raise ``NotImplementedError`` ? callers must treat
that
74         as expected and fall back to local staging."""
75         raise NotImplementedError(f"{type(self).__name__} cannot open streaming
writers")
76

```

5. user (2026-07-08T05:45:13.971Z)

```

1     """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3     URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4     lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5     ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6     today's behaviour byte-for-byte; ``gdrive`` likewise needs none ? it reads a locally
MOUNTED
7     Google Drive (Google Drive for Desktop) as plain files.
8
9     Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10    ``list_uris`` / ``get_storage``; ``local`` (and bare paths) make ``fetch`` an
identity
11    passthrough. See ``docs/DECOUPLED_COMPUTE``.
12    """
13
14    from __future__ import annotations
15
16    import os
17    import shutil
18    import tempfile
19    from typing import Any, Dict, Iterator, Optional
20
21    from backend.storage.base import StorageBackend
22    from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
23
24    __all__ = [
25        "StorageBackend",
26        "StorageRef",
27        "StorageStat",
28        "parse_uri",
29        "resolve_input_uri",
30        "get_storage",
31        "fetch",
32        "put",
33        "exists",
34        "signed_url",
35        "list_uris",
36        "open_writer",
37        "acquire_local_input",

```

```

38     "cleanup_acquired_local_input",
39     "input_is_local",
40     "resolve_input_ref",
41 ]
42
43
44 def _storage_dict(storage_config: Any) -> Dict[str, Any]:
45     """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
46     if storage_config is None:
47         return {}
48     if hasattr(storage_config, "model_dump"):
49         return storage_config.model_dump() # type: ignore[no-any-return]
50     return dict(storage_config)
51
52
53 def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
54     """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
55     config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
56     callers at the API/CLI boundary catch that and return a clean client error (never a
500).
57
58     Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
59     ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
60     sc = _storage_dict(storage_config)
61     uri = resolve_input_uri(
62         raw,
63         input_backend=sc.get("input_backend", "local"),
64         input_root=sc.get("input_root", ""),
65     )
66     return parse_uri(uri)
67
68
69 def input_is_local(raw: str, storage_config: Any = None) -> bool:
70     """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
71     plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
72     that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
73     ``resolve_input_ref``)."""
74     return resolve_input_ref(raw, storage_config).backend == "local"
75
76
77 def acquire_local_input(
78     raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
79 ) -> str:
80     """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
81
82     Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
83     byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
84     non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
85     path returned. The caller owns successful remote scratch copies; use
86     ``cleanup_acquired_local_input`` when the consumer has finished reading the returned
path."""
87     sc = _storage_dict(storage_config)
88     ref = resolve_input_ref(raw, sc)
89     if ref.backend == "local":
90         return (

```

```

91         ref.key
92     ) # identity ? no temp, no copy, byte-identical to passing the path directly
93     scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
94     try:
95         return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
96     except Exception:
97         shutil.rmtree(scratch, ignore_errors=True)
98         raise
99
100
101 def cleanup_acquired_local_input(
102     raw: str, local_path: str, storage_config: Any = None
103 ) -> None:
104     """Delete a remote scratch dir produced by ``acquire_local_input``.
105
106     Local inputs are identity paths and must never be removed. Remote acquisitions are
107     downloaded into a fresh ``rally-input-`` directory; deleting that directory is safe once the
108     caller has finished hashing/validating/stitching/segmenting the returned file.
109     """
110     try:
111         if input_is_local(raw, storage_config):
112             return
113     except ValueError:
114         return
115     if not local_path:
116         return
117     scratch = os.path.dirname(os.path.abspath(local_path))
118     if os.path.basename(scratch).startswith("rally-input-") and os.path.isdir(scratch):
119         shutil.rmtree(scratch, ignore_errors=True)
120
121
122 def _backend_class(name: str):
123     """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
124     if name == "local":
125         from backend.storage.local import LocalStorage
126
127         return LocalStorage
128     if name == "s3":
129         from backend.storage.s3 import S3Storage
130
131         return S3Storage
132     if name == "gcs":
133         from backend.storage.gcs import GCSStorage
134
135         return GCSStorage
136     if name == "gdrive":
137         from backend.storage.gdrive import GDriveStorage
138
139         return GDriveStorage
140     if name == "gdrive-api":
141         from backend.storage.gdrive_api import GDriveApiStorage
142
143         return GDriveApiStorage
144     raise ValueError(
145         f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
146     )
147
148
149 def get_storage(
150     backend: Optional[str] = None,
151     *,
152     config: Optional[Dict[str, Any]] = None,
153     **overrides,

```

```

154     ) -> StorageBackend:
155         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
156         the implementation; its per-backend settings come from ``config[backend]`` merged
with
157         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
158         cfg = config or {}
159         name = backend or cfg.get("backend", "local")
160         # Per-backend settings live under the backend name; tolerate the hyphen?underscore
form so a
161         # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
hyphen
162         # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
163         settings: Dict[str, Any] = dict(
164             cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
165         )
166         settings.update(overrides)
167         return _backend_class(name)(**settings)
168
169
170     def fetch(
171         uri: str,
172         dst: "str | os.PathLike[str] | None" = None,
173         *,
174         config: Optional[Dict[str, Any]] = None,
175         overwrite: bool = False,
176     ) -> str:
177         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
178         (no copy); for s3/gcs it downloads to ``dst``."""
179         ref = parse_uri(uri)
180         return get_storage(ref.backend, config=config).fetch_to_local(
181             ref, dst, overwrite=overwrite
182         )
183
184
185     def put(
186         local_path: "str | os.PathLike[str]",
187         uri: str,
188         *,
189         config: Optional[Dict[str, Any]] = None,
190         content_type: Optional[str] = None,
191     ) -> str:
192         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
193         ref = parse_uri(uri)
194         get_storage(ref.backend, config=config).put(
195             local_path, ref, content_type=content_type
196         )
197         return uri
198
199
200     def exists(uri: str, *, config: Optional[Dict[str, Any]] = None) -> bool:
201         """True if the object at ``uri`` exists (a bucket object or a local file)."""
202         ref = parse_uri(uri)
203         return bool(get_storage(ref.backend, config=config).exists(ref))
204
205
206     def signed_url(
207         uri: str,
208         *,
209         config: Optional[Dict[str, Any]] = None,
210         ttl: Optional[int] = None,
211         method: str = "GET",
212     ) -> str:

```

```

213         """A time-limited signed URL for ``uri`` ? the client fetches bytes DIRECTLY from
the store, off
214         the app's response path (so a large object isn't streamed back through the server;
#463). Backends
215         without object signing (e.g. local) raise ``NotImplementedError``.
216
217         ``ttl=None`` (the default) defers to the backend's configured ``signed_url_ttl``
from
218         ``config`` ? a hardcoded default here would silently shadow the operator's setting
(#469).
219         An explicit ``ttl`` overrides it for callers that need a specific expiry."""
220         ref = parse_uri(uri)
221         return get_storage(ref.backend, config=config).signed_url(ref, ttl=ttl,
method=method)
222
223
224     def list_uris(
225         prefix_uri: str, *, config: Optional[Dict[str, Any]] = None
226     ) -> Iterator[str]:
227         """Yield the URI of every object under ``prefix_uri``."""
228         ref = parse_uri(prefix_uri)
229         for r in get_storage(ref.backend, config=config).list(ref):
230             yield r.uri
231
232
233     def open_writer(
234         uri: str,
235         *,
236         config: Optional[Dict[str, Any]] = None,
237         content_type: Optional[str] = None,
238     ):
239         """A writable binary file-like streaming to ``uri`` (finalized on close). Backends
that
240         cannot stream raise ``NotImplementedError`` ? callers fall back to local staging
(see
241         ``StorageBackend.open_writer``)."""
242         ref = parse_uri(uri)
243         return get_storage(ref.backend, config=config).open_writer(
244             ref, content_type=content_type
245         )
246

```

6. user (2026-07-08T05:45:14.392Z)

```

1     """Storage value objects + URI parsing for the pluggable storage layer.
2
3     A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5     <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6     local:///<path>            -> local    (local:///abs or local://rel/p)
7     s3:///<bucket>/<key>       -> s3       (AWS + R2 + D0 Spaces + MinIO via
endpoint_url)
8     gcs:///<bucket>/<key>      -> gcs      (gs:// accepted, normalized to gcs://)
9     gdrive:///<path>           -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11     No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12     stays cheap.
13     """
14
15     from __future__ import annotations
16
17     import re

```

```

18     from dataclasses import dataclass
19     from datetime import datetime
20     from typing import Optional
21
22     _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
23     _KNOWN = ("local", "s3", "gcs", "gdrive", "gdrive-api")
24
25
26     @dataclass(frozen=True)
27     class StorageStat:
28         """Lightweight object metadata (uniform across backends)."""
29
30         size: int
31         modified: Optional[datetime] = None
32         etag: Optional[str] = None
33         content_type: Optional[str] = None
34         # Whole-object MD5 as a HEX digest ? the same value ``compute_video_id`` produces
for the
35         # same bytes, so a submit can learn a video's identity from metadata alone (#484).
Only
36         # backends that expose a server-computed MD5 fill it (GCS non-composite objects);
None
37         # means "unknown", never "no match".
38         md5: Optional[str] = None
39
40
41     @dataclass(frozen=True)
42     class StorageRef:
43         """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
44         empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
45
46         backend: str
47         bucket: str
48         key: str
49         uri: str
50
51         @property
52         def name(self) -> str:
53             return self.key.rstrip("/").rsplit("/", 1)[-1]
54
55
56     def parse_uri(uri: str) -> StorageRef:
57         """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
58
59         A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
60         have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
61         """
62         s = str(uri)
63         m = _SCHEME_RE.match(s)
64         if not m:
65             return StorageRef("local", "", s, f"local://{s}")
66         scheme = m.group(1).lower()
67         rest = s[m.end() :]
68         if scheme == "gs":
69             scheme = "gcs"
70         if scheme in ("local", "gdrive", "gdrive-api"):
71             # Path-keyed, no bucket: local FS, a path under the mounted Drive, or a path
under a Drive-API
72             # root folder (gdrive-api resolves the path ? a Drive file id).
73             return StorageRef(scheme, "", rest, s)
74         if scheme in _KNOWN:
75             bucket, _, key = rest.partition("/")

```

```

76         return StorageRef(scheme, bucket, key, s)
77         raise ValueError(f"unsupported storage URI scheme: {scheme}://{ (in {s!r})}")
78
79
80     def resolve_input_uri(
81         raw: str, *, input_backend: str = "local", input_root: str = ""
82     ) -> str:
83         """Resolve a user-supplied input reference into a storage URI
84         (COMPUTE_DECOUPLED_SERVING M2).
85
86         Precedence (first match wins):
87         1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
88         ``gdrive://``) ? as-is.
89         2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
90         local).
91         3. A bare/relative name + a non-empty ``input_root`` ? joined under it
92         (``gs://bucket/videos`` + ``x.mp4`` ? ``gs://bucket/videos/x.mp4``).
93         4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``gcs`` +
94         ``bucket/x`` ?
95         ``gcs://bucket/x``).
96         5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
97
98         So the default (``input_backend="local"`` , ``input_root=""``) is an identity
99         passthrough for every
100         path used today, preserving byte-for-byte behaviour.
101         """
102         s = str(raw)
103         if _SCHEME_RE.match(s): # 1. explicit scheme wins
104             return s
105         # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
106         INDEPENDENTLY ?
107         # os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
108         POSIX-style
109         # "/data/x" is NOT absolute). Anchored == a leading "/" or "\\ ", or a Windows
110         drive prefix "X:".
111         if s.startswith((" /", "\\ ")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
112             return s
113         if input_root: # 3. join a relative name under the configured root
114             return input_root.rstrip("/") + "/" + s.lstrip("/")
115         if (
116             input_backend and input_backend != "local"
117         ): # 4. prefix a bucket/key with the declared backend
118             return f"{input_backend}://{s}"
119         return s # 5. bare/relative local path ? unchanged

```

7. user (2026-07-08T05:45:20.073Z)

```

1     """Google Cloud Storage backend ? first-class object buckets.
2
3     Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
4     it;
5     install with the ``storage-gcs`` extra. Credentials come from Application Default
6     Credentials
7     (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
8     identity) ?
9     never from config. A ``client`` may be injected for tests.
10    """
11
12    from __future__ import annotations
13
14    import base64
15    import binascii
16    import logging
17    from datetime import timedelta

```

```

15     from importlib import import_module
16     from typing import Any, Iterator, Optional
17
18     from backend.storage.base import StorageBackend
19     from backend.storage.ref import StorageRef, StorageStat
20
21     logger = logging.getLogger(__name__)
22
23
24     def _md5_hex(md5_b64: "str | None") -> Optional[str]:
25         """GCS reports ``md5Hash`` as base64 of the raw digest; ``compute_video_id`` (the DB
key)
26         is a HEX digest ? convert so the two are directly comparable (#484). Composite
objects
27         have no md5Hash ? None; a malformed value is treated as absent, never raised."""
28         if not md5_b64:
29             return None
30         try:
31             raw = base64.b64decode(md5_b64, validate=True)
32         except (binascii.Error, ValueError):
33             return None
34         return raw.hex() if len(raw) == 16 else None
35
36
37     class GCSStorage(StorageBackend):
38         def __init__(
39             self,
40             *,
41             client: Any = None,
42             project: Optional[str] = None,
43             signed_url_ttl: int = 3600,
44             **_ignored,
45         ) -> None:
46             self._ttl = signed_url_ttl
47             if client is not None:
48                 self._client = client
49             return
50             storage_module: Any = import_module("google.cloud.storage")
51             self._client = storage_module.Client(project=project)
52
53         def _blob(self, ref: StorageRef):
54             return self._client.bucket(ref.bucket).blob(ref.key)
55
56         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
57             if dst is None:
58                 raise ValueError("GCSStorage.fetch_to_local requires a dst path")
59             import os
60
61             dst = str(dst)
62             if os.path.exists(dst) and not overwrite:
63                 return dst
64             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
65             self._blob(ref).download_to_filename(dst)
66             return dst
67
68         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
69             self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
70             return ref
71
72         def exists(self, ref) -> bool:
73             return bool(self._blob(ref).exists())
74
75         def stat(self, ref) -> StorageStat:
76             b = self._blob(ref)
77             b.reload()

```

```

78         return StorageStat(
79             size=int(b.size or 0),
80             modified=b.updated,
81             etag=b.etag,
82             content_type=b.content_type,
83             md5=_md5_hex(getattr(b, "md5_hash", None)),
84         )
85
86     def list(self, prefix) -> Iterator[StorageRef]:
87         for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
88             yield StorageRef(
89                 "gcs", prefix.bucket, blob.name, f"gcs://{prefix.bucket}/{blob.name}"
90             )
91
92     def delete(self, ref) -> None:
93         self._blob(ref).delete()
94
95     def open_writer(self, ref, *, content_type=None):
96         blob = self._blob(ref)
97         if content_type:
98             blob.content_type = content_type
99         # ignore_flush: BlobWriter only accepts flush() on chunk boundaries and raises
100         # otherwise; upload staging writes sequentially and finalizes with close(), so a
101         # caller-side flush must be a no-op, not an error. Bytes stream out per internal
102         # chunk (default 40 MiB) ? the whole object is never held in memory.
103         return blob.open("wb", ignore_flush=True)
104
105     def signed_url(self, ref, *, ttl=None, method="GET") -> str:
106         blob = self._blob(ref)
107         exp = timedelta(seconds=ttl or self._ttl)
108         creds = getattr(self._client, "_credentials", None)
109         # Local-key signing (a service-account JSON key exposes a `signer`) works out of
110         # the box. But
111         # on Cloud Run / GCE the ADC credentials come from the metadata server and have
112         # NO private key,
113         # so a plain generate_signed_url raises "you need a private key". Fall back to
114         # the IAM signBlob
115         # API ? which requires the runtime SA to hold
116         # roles/iam.serviceAccountTokenCreator on ITSELF.
117         # ? signBlob canNOT ride the storage client's own token (#475, found on the
118         # first real-creds
119         # iamcredentials rejects
120         # with ACCESS_TOKEN_SCOPE_INSUFFICIENT ? and a metadata credential may still
121         # report the
122         # "default" email alias. Mint a separate cloud-platform-scoped credential
123         # instead: its refresh
124         # both authorizes signBlob and resolves the real SA email. Failures are LOGGED
125         # before deferring
126         # to the default call, so a broken sign path can't masquerade as a missing-key
127         # error.
128         if creds is not None and not hasattr(creds, "signer"):
129             try:
130                 sa_email, token = self._signblob_identity()
131                 if sa_email:
132                     return blob.generate_signed_url(
133                         version="v4",
134                         expiration=exp,
135                         method=method,
136                         service_account_email=sa_email,
137                         access_token=token,
138                     )
139             except:
140                 logger.warning(
141                     "[gcs] keyless signing skipped: runtime SA email unresolved (got
142                     %r); "

```

```

132         "falling back to key-based signing",
133         sa_email,
134     )
135     except Exception as e:
136         logger.warning(
137             "[gcs] keyless signBlob path failed (%s); falling back to key-based
signing",
138             e,
139         )
140     return blob.generate_signed_url(version="v4", expiration=exp, method=method)
141
142     @staticmethod
143     def _signblob_identity() -> "tuple[Optional[str], Optional[str]]":
144         """(sa_email, access_token) for the IAM signBlob call ? a FRESH cloud-platform-
scoped
145         credential from ADC, never the storage client's devstorage-scoped token (#475).
Isolated
146         as a seam so tests can fake the metadata-server identity without real
credentials."""
147         import google.auth
148         from google.auth.transport.requests import Request as _AuthRequest
149
150         sign_creds, _ = google.auth.default(
151             scopes=["https://www.googleapis.com/auth/cloud-platform"]
152         )
153         sign_creds.refresh(_AuthRequest())
154         sa_email = getattr(sign_creds, "service_account_email", None)
155         if not sa_email or sa_email == "default":
156             return None, None
157         return sa_email, sign_creds.token
158

```

8. user (2026-07-08T05:45:21.066Z)

```

1     """Local-filesystem storage backend ? the default, zero new dependencies.
2
3     Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
(the
4     file is already local), ``put`` is a copy (a no-op when src == dst). With
``backend='local'``
5     the whole pipeline behaves exactly as before the storage layer existed.
6     """
7
8     from __future__ import annotations
9
10    import os
11    import shutil
12    from datetime import datetime
13    from typing import Iterator
14
15    from backend.storage.base import StorageBackend
16    from backend.storage.ref import StorageRef, StorageStat
17
18
19    class LocalStorage(StorageBackend):
20        def __init__(self, *, base_dir: str = "", **_ignored) -> None:
21            # Relative keys resolve under base_dir when set; absolute keys are used as-is.
22            self.base_dir = base_dir
23
24        def _path(self, ref: StorageRef) -> str:
25            p = ref.key
26            if self.base_dir and not os.path.isabs(p):
27                return os.path.join(self.base_dir, p)
28            return p
29

```

```

30     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31         # The object is already on the local FS ? return its path (identity). ``dst`` is
32         # honoured only if it names a *different* location (a copy is then made).
33         src = self._path(ref)
34         if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
35             return src
36         dst = str(dst)
37         if os.path.exists(dst) and not overwrite:
38             return dst
39         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
40         shutil.copyfile(src, dst)
41         return dst
42
43     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
44         dst = self._path(ref)
45         if os.path.abspath(str(local_path)) != os.path.abspath(dst):
46             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
47             shutil.copyfile(str(local_path), dst)
48         return ref
49
50     def exists(self, ref) -> bool:
51         return os.path.exists(self._path(ref))
52
53     def stat(self, ref) -> StorageStat:
54         st = os.stat(self._path(ref))
55         return StorageStat(
56             size=st.st_size, modified=datetime.fromtimestamp(st.st_mtime)
57         )
58
59     def list(self, prefix) -> Iterator[StorageRef]:
60         base = self._path(prefix)
61         if os.path.isdir(base):
62             for root, _dirs, files in os.walk(base):
63                 for fn in sorted(files):
64                     full = os.path.join(root, fn)
65                     yield StorageRef("local", "", full, f"local://{full}")
66         elif os.path.exists(base):
67             yield StorageRef("local", "", base, f"local://{base}")
68
69     def delete(self, ref) -> None:
70         path = self._path(ref)
71         if os.path.exists(path):
72             os.remove(path)
73

```

9. user (2026-07-08T05:45:21.241Z)

```

1     """S3 (and S3-compatible) storage backend ? first-class object buckets.
2
3     One backend covers **AWS S3, Cloudflare R2, DigitalOcean Spaces, and MinIO** ? they
differ
4     only by ``endpoint_url`` (+ region / addressing style). Uses ``boto3``, imported lazily
so
5     ``import backend.storage`` never pulls it; install with the ``storage-s3`` extra.
6
7     Credentials are NEVER read from config ? only ``endpoint_url`` / ``region`` / addressing
8     style. boto3's default chain supplies creds (``AWS_ACCESS_KEY_ID`` /
``AWS_SECRET_ACCESS_KEY``
9     / ``AWS_SESSION_TOKEN`` env, instance role, or ``~/.aws``). A ``client`` may be injected
10    (tests pass a fake; production lets the constructor build one via boto3).
11    """
12
13    from __future__ import annotations
14
15    import os

```

```

16     from typing import Any, Iterator, Optional
17
18     from backend.storage.base import StorageBackend
19     from backend.storage.ref import StorageRef, StorageStat
20
21
22     class S3Storage(StorageBackend):
23         def __init__(
24             self,
25             *,
26             client: Any = None,
27             endpoint_url: Optional[str] = None,
28             region: Optional[str] = None,
29             addressing_style: str = "auto",
30             signed_url_ttl: int = 3600,
31             **_ignored,
32         ) -> None:
33             self._ttl = signed_url_ttl
34             if client is not None:
35                 self._client = client
36             return
37             import boto3 # type: ignore # lazy: only when an s3:// URI is actually used
38             from botocore.config import Config # type: ignore
39
40             self._client = boto3.client(
41                 "s3",
42                 endpoint_url=endpoint_url or os.environ.get("AWS_ENDPOINT_URL_S3") or None,
43                 region_name=region,
44                 config=Config(s3={"addressing_style": addressing_style}),
45             )
46
47         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
48             if dst is None:
49                 raise ValueError("S3Storage.fetch_to_local requires a dst path")
50             dst = str(dst)
51             if os.path.exists(dst) and not overwrite:
52                 return dst
53             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
54             self._client.download_file(ref.bucket, ref.key, dst)
55             return dst
56
57         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
58             extra = {"ContentType": content_type} if content_type else None
59             self._client.upload_file(str(local_path), ref.bucket, ref.key, ExtraArgs=extra)
60             return ref
61
62         def exists(self, ref) -> bool:
63             try:
64                 self._client.head_object(Bucket=ref.bucket, Key=ref.key)
65                 return True
66             except Exception: # noqa: BLE001 - any boto/client error means "not found /
unreachable"
67                 return False
68
69         def stat(self, ref) -> StorageStat:
70             h = self._client.head_object(Bucket=ref.bucket, Key=ref.key)
71             return StorageStat(
72                 size=int(h.get("ContentLength", 0)),
73                 modified=h.get("LastModified"),
74                 etag=h.get("ETag"),
75                 content_type=h.get("ContentType"),
76             )
77
78         def list(self, prefix) -> Iterator[StorageRef]:
79             paginator = self._client.get_paginator("list_objects_v2")

```

```

80         for page in paginator.paginate(Bucket=prefix.bucket, Prefix=prefix.key):
81             for obj in page.get("Contents", []) or []:
82                 key = obj["Key"]
83                 yield StorageRef(
84                     "s3", prefix.bucket, key, f"s3://{prefix.bucket}/{key}"
85                 )
86
87     def delete(self, ref) -> None:
88         self._client.delete_object(Bucket=ref.bucket, Key=ref.key)
89
90     def signed_url(self, ref, *, ttl=None, method="GET") -> str:
91         op = "get_object" if method.upper() == "GET" else "put_object"
92         return self._client.generate_presigned_url(
93             op,
94             Params={"Bucket": ref.bucket, "Key": ref.key},
95             ExpiresIn=ttl or self._ttl,
96         )
97

```

10. user (2026-07-08T05:45:26.689Z)

```

1     """Honest, secret-free report of which storage media THIS box can use
(STUDIO_MEDIA_LIFECYCLE.md P3).
2
3     `GET /api/capabilities` surfaces this so the Studio medium picker (P5) renders only
the backends
4     this box can actually store into ? never a dead option. The contract mirrors the SPA
mock
5     (`khelsutra/web/e2e/mock-engine.mjs` CAPS.storage): a ``default_medium`` + a list of
6     ``{id, kind, label, usable, free_gb}``.
7
8     Rules (D14, D11):
9     - local ? always usable; ``free_gb`` = free space on the output filesystem (best-
effort).
10    - gcs / s3 ? usable only when the cloud SDK imports AND the box has that bucket
configured
11    (a URI-shaped root or a non-empty per-backend settings dict). ``free_gb`` is ``None``
? a bucket
12    has no readable capacity from here (cost/quota is warned elsewhere, not blocked).
13    - gdrive ? usable when a mounted Google Drive is resolvable (config / env /
autodetect);
14    ``free_gb`` = free space on that mount.
15
16    No secrets ever leave here ? only presence/usable bools, labels, and free-space
integers.
17    Best-effort + never raises: an unresolvable probe degrades a field to
``None``/``False``.
18    """
19
20    from __future__ import annotations
21
22    import importlib.util
23    import os
24    import shutil
25    from typing import Any, Dict, List, Optional
26
27    _BUCKET_LABELS = {
28        "gcs": "Google Cloud Storage bucket",
29        "s3": "Amazon S3 / compatible bucket",
30    }
31    _BUCKET_SDK = {"gcs": "google.cloud.storage", "s3": "boto3"}
32
33
34    def _as_dict(storage_config: Any) -> Dict[str, Any]:
35        if storage_config is None:

```

```

36         return {}
37     if hasattr(storage_config, "model_dump"):
38         return storage_config.model_dump() # type: ignore[no-any-return]
39     return dict(storage_config)
40
41
42     def _free_gb(path: Optional[str]) -> Optional[int]:
43         """Whole GB free on the filesystem holding ``path`` (best-effort; ``None`` if
undeterminable).
44         Walks up to the nearest existing ancestor (an output dir may not exist yet)."""
45         if not path:
46             return None
47         probe = path
48         for _ in range(64):
49             try:
50                 return int(shutil.disk_usage(probe).free / (1024**3))
51             except (OSError, ValueError):
52                 parent = os.path.dirname(probe.rstrip("/\\"))
53                 if not parent or parent == probe:
54                     return None
55                 probe = parent
56         return None
57
58
59     def _sdk_importable(module: str) -> bool:
60         """True if ``module`` is importable, WITHOUT importing it (cheap, never raises)."""
61         try:
62             return importlib.util.find_spec(module) is not None
63         except (ImportError, ValueError, ModuleNotFoundError, AttributeError):
64             return False
65
66
67     def _bucket_configured(sc: Dict[str, Any], name: str) -> bool:
68         """Does the storage config name THIS bucket backend? (No secrets ? only
scheme/selector hints.)"""
69         if sc.get("backend") == name or sc.get("input_backend") == name:
70             return True
71         if sc.get(name): # a non-empty per-backend settings dict (endpoints/regions/paths)
72             return True
73         schemes = ("gs://", "gcs://") if name == "gcs" else (f"{name}://",)
74         roots = (
75             sc.get("input_root") or "",
76             sc.get("output_root") or "",
77             sc.get("workspace_root") or "",
78         )
79         return any(r.startswith(schemes) for r in roots)
80
81
82     def _bucket_backend(name: str, sc: Dict[str, Any]) -> Dict[str, Any]:
83         usable = _sdk_importable(_BUCKET_SDK[name]) and _bucket_configured(sc, name)
84         return {
85             "id": name,
86             "kind": name,
87             "label": _BUCKET_LABELS[name],
88             "usable": usable,
89             "free_gb": None, # a bucket has no readable capacity from this box
90         }
91
92
93     def _gdrive_mount_root(sc: Dict[str, Any]) -> Optional[str]:
94         """Resolve a mounted Google Drive root (config ? env ? autodetect), or ``None``."""
95         root = (sc.get("gdrive") or {}).get("mount_root") or os.environ.get(
96             "GDRIVE_MOUNT_ROOT"
97         )
98         if root and os.path.isdir(root):

```

```

99         return root
100     try:
101         from backend.storage.gdrive import _autodetect_mount_root
102
103         auto = _autodetect_mount_root()
104     except Exception: # noqa: BLE001 - detection is best-effort; never fail
capabilities
105         auto = None
106     return auto if (auto and os.path.isdir(auto)) else None
107
108
109     def _gdrive_backend(sc: Dict[str, Any]) -> Dict[str, Any]:
110         root = _gdrive_mount_root(sc)
111         return {
112             "id": "gdrive",
113             "kind": "gdrive",
114             "label": "Google Drive folder (mounted)",
115             "usable": root is not None,
116             "free_gb": _free_gb(root),
117         }
118
119
120     def storage_capabilities(
121         storage_config: Any, *, local_free_path: Optional[str] = None
122     ) -> Dict[str, Any]:
123         """Build the ``storage`` capabilities block (see module docstring).
124         ``local_free_path`` is the
125         directory whose free space represents "this computer" (the output/workspace
126         root)."""
127         sc = _as_dict(storage_config)
128         backends: List[Dict[str, Any]] = [
129             {
130                 "id": "local",
131                 "kind": "local",
132                 "label": "This computer",
133                 "usable": True,
134                 "free_gb": _free_gb(local_free_path or "."),
135             },
136             _bucket_backend("gcs", sc),
137             _bucket_backend("s3", sc),
138             _gdrive_backend(sc),
139         ]
140     return {"default_medium": "local", "backends": backends}

```

11. user (2026-07-08T05:45:27.186Z)

```

1     """Google Drive backend ? reads Drive files through a **locally mounted** Google Drive.
2
3     We make a deliberate, baked-in assumption: **to use Google Drive files you mount Google
4     Drive
5     for Desktop**, which exposes Drive as an ordinary filesystem path. That's a fair
6     requirement ? it
7     needs no service account, no API quota, no OAuth, and Drive streams + caches bytes on
8     access. So
9     ``gdrive://`` is just a *portable, OS-agnostic alias* over that mount: it names a path
10    **relative to
11    your "My Drive" folder**, and everything else is plain local-filesystem I/O (the same
12    code the
13    ``local`` backend uses).
14
15    gdrive://Sharing/clip.mp4    ->    <mount>/My Drive/Sharing/clip.mp4
16
17    Why an alias instead of just a raw path? Portability: the mount root differs per machine
18    (Windows ``G:\My Drive``, macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My

```

```

Drive``), so the same
14     ``gdrive://`` URI resolves on any box with Drive mounted. The root is **auto-detected**,
or set it
15     explicitly via ``GDRIVE_MOUNT_ROOT`` / ``storage.gdrive.mount_root`` (a path, not a
secret).
16
17     A headless box (a GPU VM, CI) has **no** mount ? there, stage the file to a bucket and
use
18     ``gcs://`` / ``s3://``, or ``gdown <file-id>`` to a local path. This backend
intentionally does NOT
19     talk to the Drive API. When the mount (or a file in it) is missing, errors point the
user at the
20     fix: install Drive for Desktop and include the folder in syncing so it can be pulled
locally.
21     ""
22
23     from __future__ import annotations
24
25     import glob
26     import os
27     import string
28     from typing import Iterator, Optional
29
30     from backend.storage.base import StorageBackend
31     from backend.storage.local import LocalStorage
32     from backend.storage.ref import StorageRef, StorageStat
33
34     _INSTALL_URL = "https://www.google.com/drive/download/"
35
36     # Shared remedy line ? how to make a folder available locally. Drive for Desktop streams
files,
37     # but a folder only appears once it's part of your synced Drive (and a *shared* folder
must first
38     # be added to "My Drive"). The same fix resolves both "not mounted" and "mounted but
missing".
39     _SYNC_REMEDY = (
40         "make sure the folder is INCLUDED in syncing so its files can be pulled locally: in
Google "
41         "Drive for Desktop keep 'Stream files' on, and for a folder shared with you, open
Drive in the "
42         "browser, right-click it -> 'Add shortcut to Drive' so it appears under 'My Drive'.
Drive then "
43         "downloads the bytes on first access."
44     )
45
46
47     def _not_mounted_msg(tried: Optional[str]) -> str:
48         return (
49             "Google Drive is not mounted, so the gdrive:// backend cannot resolve any
path.\n"
50             f"    Remedy:\n"
51             f"    1. Install Google Drive for Desktop and sign in: {_INSTALL_URL}\n"
52             f"    2. Then {_SYNC_REMEDY}\n"
53             f"    3. If your 'My Drive' is not at the default location (Windows G:\My Drive,
macOS "
54             f"~/Library/CloudStorage/GoogleDrive-<acct>/My Drive), set GDRIVE_MOUNT_ROOT or "
55             f"storage.gdrive.mount_root to point at it (auto-detect tried: {tried!r})."
56         )
57
58
59     def _not_synced_msg(root: str, rel: str, uri: str) -> str:
60         return (
61             f"Google Drive is mounted ({root!r}) but {uri!r} is not present locally "
62             f"(expected at {os.path.join(root, *rel.split('/')!r}).\n"
63             f"    This usually means the folder is not synced yet ? {_SYNC_REMEDY}\n"

```

```

64         f" (Install/repair Drive for Desktop if needed: {_INSTALL_URL})"
65     )
66
67
68     def _autodetect_mount_root() -> Optional[str]:
69         """Find a mounted Google Drive 'My Drive' folder, or ``None``. Covers the Google
Drive for
70         Desktop defaults: macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My Drive`` and
Windows
71         ``<letter>:\My Drive`` (``G`` is the install default). Linux/other ? configure
explicitly."""
72         home = os.path.expanduser("~")
73         for p in sorted(
74             glob.glob(
75                 os.path.join(home, "Library", "CloudStorage", "GoogleDrive*", "My Drive")
76             )
77         ):
78             if os.path.isdir(p):
79                 return p
80         if os.name == "nt":
81             # G is the Drive-for-Desktop default; then scan the rest of the assignable
letters.
82             for letter in ["G"] + [c for c in string.ascii_uppercase if c != "G"]:
83                 p = f"{letter}:\My Drive"
84                 if os.path.isdir(p):
85                     return p
86         return None
87
88
89     class GDriveStorage(StorageBackend):
90         def __init__(self, *, mount_root: Optional[str] = None, **_ignored) -> None:
91             root = (
92                 mount_root
93                 or os.environ.get("GDRIVE_MOUNT_ROOT")
94                 or _autodetect_mount_root()
95             )
96             if not root or not os.path.isdir(root):
97                 raise RuntimeError(_not_mounted_msg(root))
98             self._root = os.path.abspath(root)
99             self._local = LocalStorage(
100                 base_dir=self._root
101             ) # all I/O is plain local-FS under the mount
102
103         def _local_ref(self, ref: StorageRef) -> StorageRef:
104             """Translate a ``gdrive://<rel>`` ref into a ``local`` ref keyed by an OS path
relative to
105             the mount root (``LocalStorage`` joins it under ``base_dir``)."""
106             rel = ref.key.lstrip("/")
107             key = os.path.join(*rel.split("/")) if rel else ""
108             return StorageRef("local", "", key, f"local://{key}")
109
110         def _require_present(self, ref: StorageRef) -> None:
111             """Guard the read paths: a path missing from the mount almost always means the
folder
112             isn't synced ? raise with that remedy instead of a cryptic FileNotFoundError
downstream."""
113             rel = ref.key.lstrip("/")
114             abs_path = os.path.join(self._root, *rel.split("/")) if rel else self._root
115             if not os.path.exists(abs_path):
116                 raise FileNotFoundError(_not_synced_msg(self._root, rel, ref.uri))
117
118         def _to_gdrive_uri(self, abs_path: str) -> str:
119             """The inverse: an absolute path under the mount ? its ``gdrive://`` URI."""
120             rel = os.path.relpath(abs_path, self._root).replace(os.sep, "/")
121             return f"gdrive://{rel}"

```

```

122
123     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
124         # The file is already local (via the mount): no dst => identity passthrough to
the mount
125         # path; a dst => copy out (a byte-stable scratch copy), exactly as the local
backend does.
126         self._require_present(ref)
127         return self._local.fetch_to_local(
128             self._local_ref(ref), dst, overwrite=overwrite
129         )
130
131     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
132         self._local.put(local_path, self._local_ref(ref), content_type=content_type)
133         return ref
134
135     def exists(self, ref) -> bool:
136         return self._local.exists(self._local_ref(ref))
137
138     def stat(self, ref) -> StorageStat:
139         self._require_present(ref)
140         return self._local.stat(self._local_ref(ref))
141
142     def list(self, prefix) -> Iterator[StorageRef]:
143         for r in self._local.list(self._local_ref(prefix)):
144             uri = self._to_gdrive_uri(r.key)
145             yield StorageRef("gdrive", "", uri[len("gdrive://") :], uri)
146
147     def delete(self, ref) -> None:
148         self._local.delete(self._local_ref(ref))
149

```

12. user (2026-07-08T05:45:27.710Z)

```

1     """Google Drive **API** backend (OAuth / service-account) ? ``gdrive-api://`` (M12, D14
Northstar).
2
3     The cloud-native Drive round-trip: read a source from + write the stitched reel **back
to** a user's
4     Google Drive **via the Drive API** ? no local mount required (unlike the ``gdrive://``
backend, which
5     is plain local-FS over a *mounted* Drive for Desktop and stays untouched). This is the
headless path a
6     Cloud Run / GPU box uses.
7
8     ``gdrive-api://<path>`` names a path **relative to a root folder** (default the Drive
``root``, or
9     ``storage.gdrive-api.root_folder_id``); the backend resolves each path component to a
Drive file id via
10    ``files().list``. The Google libraries (``google-api-python-client`` + ``google-auth``)
are imported
11    **lazily** ? ``import backend.storage`` never pulls them ? and a ``service`` may be
**injected** for
12    fully-offline tests (mirroring ``GCSStorage(client=...)``).
13
14    ? **Credentials are never in config.** The real service is built from Application
Default Credentials
15    (a service account / workload identity) over the ``drive`` scope. **Per-user interactive
OAuth (the
16    consumer "connect your Drive" flow: client id/secret + refresh tokens) is OWNER-gated**
? this backend
17    ships the service-account/ADC build + the injected fake; the live per-user OAuth app is
a follow-up.
18
19    **Scope caveats (M12):** handles **binary** files (videos / reels / JSON) ? Google-
native Docs

```

```

20     (Sheets/Slides) have no direct download and would need ``files().export_media`` (out of
scope; export
21     them first). ``put`` is idempotent: an existing file at the same path is overwritten
**in place** via
22     ``files().update`` (same file id; the old bytes survive until the new upload succeeds ?
never
23     delete-then-create). Path resolution keys on ``(name, parent)``; if Drive already holds
24     duplicate-named files under one folder (a rare pre-existing state this backend never
creates),
25     resolution returns the first match. Downloads stream in chunks (``MediaIoBaseDownload``)
? a
26     multi-GB reel is never buffered whole in RAM.
27     ""
28
29     from __future__ import annotations
30
31     import io
32     import os
33     from datetime import datetime
34     from typing import Any, Iterator, Optional
35
36     from backend.storage.base import StorageBackend
37     from backend.storage.ref import StorageRef, StorageStat
38
39     _FOLDER_MIME = "application/vnd.google-apps.folder"
40     _DRIVE_SCOPE = "https://www.googleapis.com/auth/drive"
41     # Download chunk size: bounds fetch_to_local RAM no matter the object size (vs. the
googleapiclient
42     # default of 100 MiB) while keeping the number of round-trips per multi-GB reel
reasonable.
43     _DOWNLOAD_CHUNK_BYTES = 32 * 1024 * 1024
44
45
46     def _q_escape(name: str) -> str:
47         """Escape a value for a Drive ``q`` string literal (single-quoted): ``\`` then
``'``, ""
48         return name.replace("\", "\\").replace("'", "\\'")
49
50
51     class GDriveApiStorage(StorageBackend):
52         """Drive-API-backed storage. Construct with an injected ``service`` (tests) or let
it build one
53         from ADC (production). ``root_folder_id`` anchors relative ``gdrive-api://``
paths."""
54
55         def __init__(
56             self, *, service: Any = None, root_folder_id: str = "root", **_ignored
57         ) -> None:
58             self._root_id = root_folder_id or "root"
59             if service is not None:
60                 self._service = service
61                 return
62             self._service = self._build_service()
63
64         # -- construction (real path; lazy Google imports)
-----
65         def _build_service(self) -> Any:
66             try:
67                 import google.auth # type: ignore
68                 from googleapiclient.discovery import build # type: ignore
69             except (
70                 ImportError
71             ) as e: # pragma: no cover - exercised only without the extra installed
72                 raise RuntimeError(
73                     "the gdrive-api backend needs the 'storage-gdrive-api' extra "

```

```

74         "(google-api-python-client + google-auth). Install it on a host that
uses "
75         "gdrive-api:// URIs; credentials come from ADC (a service account /
workload "
76         "identity over the Drive scope) ? never from config."
77     ) from e
78     creds, _ = google.auth.default(scopes=[_DRIVE_SCOPE])
79     return build("drive", "v3", credentials=creds, cache_discovery=False)
80
81     def _media_body(self, local_path: str, content_type: Optional[str]) -> Any:
82         """The upload media object for ``files().create`` ? lazy ``MediaFileUpload``
(real path).
83         A test seam: monkeypatch this to avoid the ``googleapiclient`` dependency."""
84         from googleapiclient.http import (
85             MediaFileUpload, # type: ignore # pragma: no cover
86         )
87
88         return MediaFileUpload(
89             local_path, mimetype=content_type, resumable=True
90         ) # pragma: no cover
91
92     def _downloader(self, fh: Any, request: Any) -> Any:
93         """The chunked-download driver for a ``files().get_media`` request ? lazy
``MediaIoBaseDownload`` (real path). A test seam: monkeypatch this to avoid the
94         ``googleapiclient`` dependency (the fakes drive the same ``next_chunk()``
95         loop)."""
96         from googleapiclient.http import (
97             MediaIoBaseDownload, # type: ignore # pragma: no cover
98         )
99
100         return MediaIoBaseDownload(
101             fh, request, chunksize=_DOWNLOAD_CHUNK_BYTES
102         ) # pragma: no cover
103
104     # -- path -> id resolution
-----
105     def _resolve_id(self, key: str, *, create_dirs: bool = False) -> Optional[str]:
106         """Walk ``key``'s path components from the root folder, returning the leaf's
file id (or
107         ``None`` if a component is missing). With ``create_dirs`` EVERY missing
component is created as
108         a folder ? ``put`` uses this on the parent-directory path (where all components
are folders),
109         so a write into a fresh subfolder works. The read paths leave it ``False`` (a
missing component
110         ? ``None``), never creating anything."""
111         parent = self._root_id
112         parts = [p for p in key.split("/") if p]
113         for part in parts:
114             q = (
115                 f"name = '{_q_escape(part)}' and '{_q_escape(parent)}' in parents "
116                 f"and trashed = false"
117             )
118             found = (
119                 self._service.files()
120                 .list(q=q, spaces="drive", fields="files(id, mimeType)")
121                 .execute()
122                 .get("files", [])
123             )
124             if found:
125                 parent = found[0]["id"]
126             elif create_dirs:
127                 parent = (
128                     self._service.files()
129                     .create(

```

```

130         body={
131             "name": part,
132             "mimeType": _FOLDER_MIME,
133             "parents": [parent],
134         },
135         fields="id",
136     )
137     .execute()["id"]
138 )
139 else:
140     return None
141 return parent
142
143 def _split(self, key: str) -> tuple[str, str]:
144     """Split a ``gdrive-api://`` key into (parent dir path, leaf name)."""
145     clean = key.strip("/")
146     head, _, name = clean.rpartition("/")
147     return head, name
148
149 # -- StorageBackend interface
-----
150 def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
151     if dst is None:
152         raise ValueError(
153             "GDriveApiStorage.fetch_to_local requires a dst path (no local mount)"
154         )
155     dst = str(dst)
156     if os.path.exists(dst) and not overwrite:
157         return dst
158     file_id = self._resolve_id(ref.key)
159     if not file_id:
160         raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
161     request = self._service.files().get_media(fileId=file_id) # alt=media ? bytes
162     os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
163     # Stream in bounded chunks to a ``.part`` sibling, then rename into place: a
multi-GB
164     # reel never sits whole in RAM (the old ``get_media().execute()`` buffered the
entire
165     # object as one bytes value ? OOM fodder on a memory-limited Cloud Run box), and
a
166     # mid-download failure can't leave a truncated ``dst`` that a later
``overwrite=False``
167     # call would trust as complete.
168     tmp = dst + ".part"
169     try:
170         with io.open(tmp, "wb") as fh:
171             downloader = self._downloader(fh, request)
172             done = False
173             while not done:
174                 _, done = downloader.next_chunk()
175             os.replace(tmp, dst)
176     finally:
177         if os.path.exists(tmp):
178             os.remove(tmp)
179     return dst
180
181 def put(self, local_path, ref, *, content_type=None) -> StorageRef:
182     parent_path, name = self._split(ref.key)
183     parent_id = (
184         self._resolve_id(parent_path, create_dirs=True)
185         if parent_path
186         else self._root_id
187     )
188     if parent_id is None:
189         raise FileNotFoundError(f"gdrive-api: cannot resolve parent of {ref.uri!r}")

```

```

190         # Idempotent: Drive allows duplicate (name, parent), so a plain create() on a
re-put would
191         # ORPHAN the prior file (and reads would keep returning the stale first match).
Overwrite
192         # an existing file at this exact path IN PLACE via files().update ? the file id
is kept
193         # and the OLD bytes survive until the NEW upload succeeds. (The previous delete-
then-create
194         # ordering lost the object outright when the create failed mid-upload; create-
first-then-
195         # delete would instead open a duplicate-(name, parent) window where reads could
resolve
196         # either copy.)
197         media = self._media_body(str(local_path), content_type)
198         existing = self._resolve_id(ref.key)
199         if existing:
200             self._service.files().update(fileId=existing, media_body=media).execute()
201         else:
202             self._service.files().create(
203                 body={"name": name, "parents": [parent_id]},
204                 media_body=media,
205                 fields="id",
206             ).execute()
207         return ref
208
209     def exists(self, ref) -> bool:
210         return self._resolve_id(ref.key) is not None
211
212     def stat(self, ref) -> StorageStat:
213         file_id = self._resolve_id(ref.key)
214         if not file_id:
215             raise FileNotFoundError(f"gd drive-api: {ref.uri!r} not found")
216         meta = (
217             self._service.files()
218             .get(fileId=file_id, fields="size, modifiedTime, md5Checksum, mimeType")
219             .execute()
220         )
221         modified = None
222         if meta.get("modifiedTime"):
223             # Drive returns RFC-3339 (e.g. "2026-06-22T10:00:00.000Z"); normalize the
trailing Z.
224             modified = datetime.fromisoformat(
225                 meta["modifiedTime"].replace("Z", "+00:00")
226             )
227         return StorageStat(
228             size=int(meta.get("size") or 0),
229             modified=modified,
230             etag=meta.get("md5Checksum"),
231             content_type=meta.get("mimeType"),
232         )
233
234     def list(self, prefix) -> Iterator[StorageRef]:
235         folder_id = (
236             self._resolve_id(prefix.key) if prefix.key.strip("/") else self._root_id
237         )
238         if not folder_id:
239             return
240         base = prefix.key.strip("/")
241         page_token: Optional[str] = None
242         while True:
243             resp = (
244                 self._service.files()
245                 .list(
246                     q=f'{"{_q_escape(folder_id)}" in parents and trashed = false',
247                     spaces="drive",

```

```

248             fields="nextPageToken, files(id, name)",
249             pageToken=page_token,
250         )
251         .execute()
252     )
253     for f in resp.get("files", []):
254         key = f"{base}/{f['name']}" if base else f["name"]
255         yield StorageRef("gdrive-api", "", key, f"gdrive-api://{key}")
256     page_token = resp.get("nextPageToken")
257     if not page_token:
258         break
259
260     def delete(self, ref) -> None:
261         file_id = self._resolve_id(ref.key)
262         if file_id:
263             self._service.files().delete(fileId=file_id).execute()
264

```

13. assistant (2026-07-08T05:45:34.549Z)

Now let me find the callers and config knobs.

14. user (2026-07-08T05:45:36.724Z)

```

backend\config\models.py:431:    # scratch dir before processing (see
``backend.storage.acquire_local_input``).
backend\main.py:639:        input_ref = storage.resolve_input_ref(
backend\main.py:786:        src_ref = storage.resolve_input_ref(req.uploaded_path,
storage_settings)
backend\main.py:798:        src_ref = storage.resolve_input_ref(req.source_uri,
storage_settings)
backend\main.py:806:        local_working = storage.acquire_local_input(
backend\main.py:899:        storage.cleanup_acquired_local_input(
backend\main.py:935:        input_ref = storage.resolve_input_ref(
backend\main.py:1499:        input_ref = storage.resolve_input_ref(args.video_path,
storage_cfg)
backend\main.py:1506:        local_video = storage.acquire_local_input(args.video_path,
storage_cfg)
backend\main.py:1590:        storage.cleanup_acquired_local_input(
backend\orchestration.py:88:        if storage.input_is_local(raw_input, storage_config):
backend\orchestration.py:518:        input_ref = storage.resolve_input_ref(req.video_path,
storage_cfg)
backend\orchestration.py:576:        input_ref = storage.resolve_input_ref(staged_uri,
storage_cfg)
backend\orchestration.py:748:        local_video = storage.acquire_local_input(
backend\orchestration.py:753:        storage.cleanup_acquired_local_input(
backend\orchestration.py:760:        storage.cleanup_acquired_local_input(
backend\orchestration.py:1000:        storage.cleanup_acquired_local_input(
backend\orchestration.py:1159:        local_src = storage.acquire_local_input(
backend\orchestration.py:1177:        storage.cleanup_acquired_local_input(
backend\api\uploads.py:10:    through a ``storage.open_writer`` into
``<input_root>/uploads/<upload_id>/<name>``
backend\api\uploads.py:177:        writer = storage.open_writer(stage_uri,
config=staging["config"])
backend\storage\__init__.py:36:    "open_writer",
backend\storage\__init__.py:37:    "acquire_local_input",
backend\storage\__init__.py:38:    "cleanup_acquired_local_input",
backend\storage\__init__.py:39:    "input_is_local",
backend\storage\__init__.py:40:    "resolve_input_ref",
backend\storage\__init__.py:53:    def resolve_input_ref(raw: str, storage_config: Any = None) ->
StorageRef:
backend\storage\__init__.py:69:    def input_is_local(raw: str, storage_config: Any = None) -> bool:
backend\storage\__init__.py:73:        ``resolve_input_ref``)."""
backend\storage\__init__.py:74:        return resolve_input_ref(raw, storage_config).backend ==
"local"

```

```

backend\storage\__init__.py:77: def acquire_local_input(
backend\storage\__init__.py:86:     ``cleanup_acquired_local_input`` when the consumer has
finished reading the returned path."""
backend\storage\__init__.py:88:     ref = resolve_input_ref(raw, sc)
backend\storage\__init__.py:101: def cleanup_acquired_local_input(
backend\storage\__init__.py:104:     """Delete a remote scratch dir produced by
``acquire_local_input``.
backend\storage\__init__.py:111:         if input_is_local(raw, storage_config):
backend\storage\__init__.py:233: def open_writer(
backend\storage\__init__.py:241:     ``StorageBackend.open_writer``)."""
backend\storage\__init__.py:243:     return get_storage(ref.backend, config=config).open_writer(
backend\storage\gcs.py:95:     def open_writer(self, ref, *, content_type=None):
backend\storage\base.py:66:     def open_writer(

```

15. user (2026-07-08T05:45:38.320Z)

```

backend\config\models.py:413: class StorageConfig(BaseModel):
backend\config\models.py:423:     output_root: str = ""
backend\config\models.py:424:     # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the
output `backend`/`output_root`).
backend\config\models.py:427:     # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
backend\config\models.py:428:     # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is
empty, prefixed with ``input_backend``
backend\config\models.py:432:     input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-
api"] = "local"
backend\config\models.py:433:     input_root: str = ""
backend\config\models.py:440:     # Durable workspace root URI. "" => precedence: output_root,
then output.output_dir.
backend\config\models.py:441:     workspace_root: str = ""
backend\config\presets.py:41: # these bundles ? ``input_backend`` (M2) and the configurable
output/workspace base (M3,

```

16. user (2026-07-08T05:45:45.430Z)

```

400         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
401         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
402         # application's audience tag. NO secrets here ? these are public identifiers.
403         edge_auth_enabled: bool = False
404         cf_team_domain: str = (
405             "" # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
406         )
407         cf_access_aud: str = "" # the CF Access application AUD tag the token must carry
408         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
409         # backend/main.py so importing the app does no filesystem I/O; default ["*"].
Credentials stay
410         # OFF ? Cf-Access is a header, not a cookie.)
411
412
413     class StorageConfig(BaseModel):
414         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
415         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
416         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
417         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
418         (boto3/google auth chains supply credentials). See ``backend/storage/``.
419
420
421         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"

```

```

422         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
423         output_root: str = ""
424         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
425         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
426         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
427         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
428         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
429         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
430         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
431         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
432         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
433         input_root: str = ""
434         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
435         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
436         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
437         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
438         # the legacy flat layout is used unchanged.
439         single_folder_per_video: bool = False
440         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
441         workspace_root: str = ""
442         # Cloud namespace prefix under the root so per-video folders sit tidily beside
443         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
444         workspace_prefix: str = "workspaces"
445         local: dict[str, Any] = Field(default_factory=dict)
446         s3: dict[str, Any] = Field(default_factory=dict)
447         gcs: dict[str, Any] = Field(default_factory=dict)
448         gdrive: dict[str, Any] = Field(default_factory=dict)
449         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
450         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
451         gdrive_api: dict[str, Any] = Field(default_factory=dict)
452
453
454     class DeploymentConfig(BaseModel):
455         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
456         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
457
458         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
459         behaves exactly as before this section existed. A profile is opt-in (``config.json``
``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
460         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
461         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
462         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
463
464         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
465
466

```

```

467     # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
468     profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
469     # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
470     # (recorded for observability; consumed by the ComputeBackend seam from M2+).
471     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
472     # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
473     # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
474     # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
475     # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
476     # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
477     max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
478     # Poll budget for one dispatched remote job (CONTROL_PLANE_DURABLE_REBUILD $4.3).
Must sit
479     # ABOVE the worker Job's own --task-timeout (3600s) so the worker's timeout stays
the
480     # authority and the dispatcher's one-shot post-timeout marker rescue still applies ?
the old
481     # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
482     # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
483     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
484
485
486     class BillingConfig(BaseModel):
487         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
488
489         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
490         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
491         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
492         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
493         inserts a real, calibrated version and points ``pricebook_version`` at it.
494
495         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
496         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
497         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
498
499         enabled: bool = False
500         pricebook_version: str = (
501             "placeholder-v0" # the seeded $0 version; owner flips to a real one
502         )
503         fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore market)
504         margin: float = 0.0 # resale markup (?0); 0 = bill at cost
505
506
507     class FlywheelConfig(BaseModel):
508         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
509         loop: on a CONSENTED adult job, durably **capture** the source + rally output into
the per-video

```

```

1  """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3  Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4  proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2). Where they assemble depends
5  on the input backend (CONTROL_PLANE_DURABLE_REBUILD P2):
6
7  * ``storage.input_backend == "local"`` (appliance/default): parts append to a partial
8  file under security.upload_dir, exactly as before.
9  * a remote input backend with an ``input_root`` (the hosted alpha: gcs): parts STREAM
10 through a ``storage.open_writer`` into ``<input_root>/uploads/<upload_id>/<name>``
11 while an incremental MD5 accumulates ? the assembled file never touches local
12 disk/tmpfs. The 4 GiB RAM-``/tmp`` control plane OOM'd on a 2.68 GB browser upload
13 (#480/#471); after this, instance memory bounds a CHUNK, not the video. ``complete``
14 returns the staged URI (inside the #465 input jail by construction ? it is under
15 ``input_root``) plus the streamed md5 as ``video_id``.
16
17 Upload state is in-process by design (this is the single-box alpha): a server restart
18 aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
19 arrives with PR-3 identity scoping.
20
21 The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
22 ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
23 reads the live value the server set at startup (and so tests that monkeypatch
24 ``backend.main.global_config`` flow through unchanged).
25 """
26
27 import hashlib
28 import os
29 import threading
30 import unicodedata
31 import uuid
32 from typing import Any, Dict, Optional
33
34 from fastapi import APIRouter, HTTPException, Request
35 from pydantic import BaseModel
36 from starlette.concurrency import run_in_threadpool
37
38 from backend import storage
39 from backend.utils.app_home import (
40     resolve_output_dir, # P0a: app-home-aware path resolution
41 )
42 from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
43 from backend.utils.hashing import compute_video_id # canonical file-identity hashing
44 from backend.utils.paths import safe_output_filename
45
46 router = APIRouter()
47
48 _uploads: Dict[str, Dict[str, Any]] = {}
49 _uploads_lock = threading.Lock()
50
51
52 class UploadInitRequest(BaseModel):
53     filename: str
54     total_size_bytes: Optional[int] = None
55
56
57 class UploadCompleteRequest(BaseModel):
58     total_parts: int
59
60
61 def _upload_cfg():
62     # Resolved lazily (import inside the function) to read the LIVE startup config that
63     # main() sets on the module global, and to avoid a circular import at module load
64     # (main.py imports this router; this module must not import main at import time).
65     import backend.main as _main

```

```

66
67     sec = getattr(_main.global_config, "security", None)
68     upload_dir = resolve_output_dir(
69         getattr(sec, "upload_dir", None) or "output/uploads"
70     ) # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
71     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
72     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
73     exts = [
74         e.lower().lstrip(".")
75         for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
76     ]
77     return upload_dir, max_bytes, part_bytes, exts
78
79
80 def upload_caps(cfg) -> Dict[str, int]:
81     """The upload limits /api/capabilities advertises (#480 item 5, CP-rebuild P5a).
82
83     ``max_upload_mb`` is what the SPA's pre-flight guard rejects oversize files against
84     ?
85     without it the guard is dead code and the user learns from a mid-upload 413.
86     ``max_part_mb`` is informational: each /api/upload/init response carries the
87     authoritative value the client actually chunks by."""
88     sec = getattr(cfg, "security", None)
89     return {
90         "max_upload_mb": int(getattr(sec, "max_upload_mb", 4096) or 4096),
91         "max_part_mb": int(getattr(sec, "max_part_mb", 95) or 95),
92     }
93
94 def _staging_target() -> "Optional[Dict[str, Any]]":
95     """The remote staging root + storage settings, or None for the local-file flow.
96
97     Remote staging is selected by the same input config /api/process resolves against
98     (storage.input_backend + input_root), so a staged upload is submittable verbatim.
99     """
100     import backend.main as _main
101
102     sc = getattr(_main.global_config, "storage", None)
103     if sc is None:
104         return None
105     input_backend = getattr(sc, "input_backend", "local") or "local"
106     input_root = (getattr(sc, "input_root", "") or "").rstrip("/")
107     if input_backend == "local" or not input_root:
108         return None
109     cfg = sc.model_dump() if hasattr(sc, "model_dump") else dict(sc)
110     return {"root": input_root, "config": cfg}
111
112
113 def _abort_upload(upload_id: str) -> None:
114     with _uploads_lock:
115         st = _uploads.pop(upload_id, None)
116         if not st:
117             return
118         writer = st.get("writer")
119         if writer is not None:
120             # A streaming writer has no cancel ? close() finalizes whatever was staged, then
121             # the partial object is deleted. Both are best-effort: the upload is already
122             # aborted from the client's point of view.
123             try:
124                 writer.close()
125             except Exception: # noqa: BLE001 ? abort cleanup must never mask the abort
126                 pass
127             try:
128                 ref = storage.parse_uri(st["stage_uri"])
129                 storage.get_storage(ref.backend, config=st.get("config")).delete(ref)

```

```

130         except Exception: # noqa: BLE001
131             pass
132         return
133     try:
134         os.remove(st["tmp_path"])
135     except OSError:
136         pass
137
138
139     def _normalize_upload_filename(filename: str) -> str:
140         """Normalize browser-provided file names before extension validation.
141
142         Some mobile/file-provider pickers can hand us visually normal names with
143         compatibility
144         characters (for example a fullwidth dot) or trailing whitespace/dots. Keep the
145         existing
146         traversal/null-byte checks in ``safe_output_filename`` as the authority after this
147         cleanup.
148         """
149         return unicodedata.normalize("NFKC", filename).strip().rstrip(". ")
150
151     @router.post("/api/upload/init")
152     def upload_init(req: UploadInitRequest):
153         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
154         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
155         try:
156             name = safe_output_filename(_normalize_upload_filename(req.filename))
157         except ValueError as e:
158             raise HTTPException(status_code=400, detail=str(e)) from e
159         ext = os.path.splitext(name)[1].lower().lstrip(".")
160         if ext not in exts:
161             raise HTTPException(
162                 status_code=400,
163                 detail=f"Extension '{ext}' not allowed. Allowed: {sorted(exts)}",
164             )
165         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
166             raise HTTPException(
167                 status_code=413,
168                 detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
169             )
170         upload_id = uuid.uuid4().hex
171         staging = _staging_target()
172         if staging is not None:
173             # Remote staging: stream parts straight into the bucket under the input root ?
174             # never assembled on local disk/tmpfs (CONTROL_PLANE_DURABLE_REBUILD P2). The
175             # upload_id in the key keeps concurrent same-name uploads distinct, and the
176             free-
177             # space guard is skipped: no local bytes are consumed.
178             stage_uri = f"{staging['root']}/uploads/{upload_id}/{name}"
179             try:
180                 writer = storage.open_writer(stage_uri, config=staging["config"])
181             except NotImplementedError:
182                 staging = None # backend can't stream (e.g. mounted-Drive) ? stage locally
183             else:
184                 with _uploads_lock:
185                     _uploads[upload_id] = {
186                         "filename": name,
187                         "writer": writer,
188                         "hasher": hashlib.md5(),
189                         "stage_uri": stage_uri,
190                         "config": staging["config"],
191                         "parts": 0,
192                         "bytes": 0,
193                     }

```

```

191         return {
192             "upload_id": upload_id,
193             "max_part_mb": part_bytes // (1024 * 1024),
194             "next_part": f"/api/upload/{upload_id}/part/0",
195         }
196     os.makedirs(upload_dir, exist_ok=True)
197     # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
198     # effort ? an
199     # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
200     # (never blocks a
201     # legit upload); the per-part total cap in upload_part is the hard backstop either
202     way.
203     require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
204     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
205     open(tmp_path, "wb").close()
206     with _uploads_lock:
207         _uploads[upload_id] = {
208             "filename": name,
209             "tmp_path": tmp_path,
210             "parts": 0,
211             "bytes": 0,
212         }
213     return {
214         "upload_id": upload_id,
215         "max_part_mb": part_bytes // (1024 * 1024),
216         "next_part": f"/api/upload/{upload_id}/part/0",
217     }
218
219 @router.put("/api/upload/{upload_id}/part/{index}")
220 async def upload_part(upload_id: str, index: int, request: Request):
221     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
222     _, max_bytes, part_bytes, _ = _upload_cfg()
223     with _uploads_lock:
224         st = _uploads.get(upload_id)
225         if st is None:
226             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
227         if index != st["parts"]:
228             raise HTTPException(
229                 status_code=409,
230                 detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
231                 sequential)",
232             )
233
234     received = 0
235     overflow = None
236     if st.get("writer") is not None:
237         # Remote staging: buffer THIS part (? max_part_mb ? the caps above still abort
238         # an
239         # oversize stream), then hand it to the blocking bucket writer in one threadpool
240         # hop so a network flush never stalls the event loop mid-upload.
241         buf = bytearray()
242         async for chunk in request.stream():
243             received += len(chunk)
244             if received > part_bytes:
245                 overflow = (
246                     f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
247                 )
248                 break
249             if st["bytes"] + received > max_bytes:
250                 overflow = (
251                     f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
252                 )
253                 break
254             buf += chunk

```

```

251         if not overflow and buf:
252             data = bytes(buf)
253             await run_in_threadpool(st["writer"].write, data)
254             st["hasher"].update(data)
255     else:
256         with open(st["tmp_path"], "ab") as fh:
257             async for chunk in request.stream():
258                 received += len(chunk)
259                 if received > part_bytes:
260                     overflow = (
261                         f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
262                     )
263                     break
264                 if st["bytes"] + received > max_bytes:
265                     overflow = (
266                         f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
267                     )
268                     break
269                 fh.write(chunk)
270     if overflow:
271         _abort_upload(upload_id)
272         raise HTTPException(
273             status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
274         )
275
276     with _uploads_lock:
277         st["parts"] += 1
278         st["bytes"] += received
279     return {
280         "upload_id": upload_id,
281         "received_parts": st["parts"],
282         "received_bytes": st["bytes"],
283     }
284
285
286     @router.post("/api/upload/{upload_id}/complete")
287     def upload_complete(upload_id: str, req: UploadCompleteRequest):
288         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
289         upload_dir, _, _, _ = _upload_cfg()
290         with _uploads_lock:
291             st = _uploads.get(upload_id)
292         if st is None:
293             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
294         if st["parts"] == 0 or st["parts"] != req.total_parts:
295             raise HTTPException(
296                 status_code=409,
297                 detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
298             )
299         if st.get("writer") is not None:
300             # Remote staging: close() finalizes the streamed object; the md5 accumulated per
301             # part IS compute_video_id's digest (same bytes, same algorithm), so
/api/process
302             # gets the identity without anyone re-reading the video. (Sync endpoint ?
FastAPI
303             # runs it on the threadpool, so the blocking finalize doesn't stall the loop.)
304             try:
305                 st["writer"].close()
306             except Exception as e: # noqa: BLE001 ? surface a clean verdict, then clean up
307                 _abort_upload(upload_id)
308                 raise HTTPException(
309                     status_code=502,
310                     detail=f"Failed to finalize the staged upload ({e}). Re-init to retry.",

```

```

311         ) from e
312     with _uploads_lock:
313         _uploads.pop(upload_id, None)
314     return {
315         "path": st["stage_uri"],
316         "video_id": st["hasher"].hexdigest(),
317         "size_bytes": st["bytes"],
318         "next": "POST /api/process with this path",
319     }
320     final_path = os.path.join(upload_dir, st["filename"])
321     if os.path.exists(
322         final_path
323     ): # never overwrite an earlier upload with the same name
324         stem, ext = os.path.splitext(st["filename"])
325         final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
326     os.replace(st["tmp_path"], final_path)
327     with _uploads_lock:
328         _uploads.pop(upload_id, None)
329     video_id = compute_video_id(final_path)
330     return {
331         "path": os.path.abspath(final_path),
332         "video_id": video_id,
333         "size_bytes": os.path.getsize(final_path),
334         "next": "POST /api/process with this path",
335     }
336
337
338 @router.delete("/api/upload/{upload_id}")
339 def upload_abort(upload_id: str):
340     """Abort an in-flight upload and delete its partial file."""
341     with _uploads_lock:
342         known = upload_id in _uploads
343     if not known:
344         raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
345     _abort_upload(upload_id)
346     return {"status": "aborted", "upload_id": upload_id}
347

```

18. assistant (2026-07-08T05:45:52.418Z)

Now let me look at the key call sites in orchestration.py and main.py where storage crosses into the pipeline.

19. user (2026-07-08T05:45:54.288Z)

```

60     fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
61     and
62     keep the prior good results intact ? a failed/empty re-run never destroys prior
63     data.
64     """
65     if segments:
66         db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
67     else:
68         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
69
70 def _validation_failure_message(failures: List[Any]) -> str:
71     details = []
72     for failure in failures:
73         name = getattr(failure, "validator_name", None) or "validation"
74         message = (getattr(failure, "message", None) or "").strip()
75         details.append(f"{name}: {message}" if message else str(name))
76     if not details:
77         return "Video failed validation checks."
78     return "Video failed validation: " + "; ".join(details)

```

```

78
79
80     def _report_config_for_input(config: Any, raw_input: str) -> Any:
81         """Return report config that keeps remote-input reports out of disposable
scratch."""
82         storage_config = (
83             config.get("storage")
84             if isinstance(config, dict)
85             else getattr(config, "storage", None)
86         )
87         try:
88             if storage.input_is_local(raw_input, storage_config):
89                 return config
90         except ValueError:
91             return config
92         try:
93             cfg = (
94                 config.model_dump(mode="json")
95                 if hasattr(config, "model_dump")
96                 else dict(config or {})
97             )
98         except Exception:
99             cfg = {}
100         report = dict(cfg.get("report") or {})
101         if report.get("output_dir"):
102             return config
103         report["output_dir"] = os.path.join(output_base(config), "reports")
104         cfg["report"] = report
105         return cfg
106
107
108     def _ingest_remote_segments(
109         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
110     ) -> int:
111         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
112         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
113         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
114         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
115         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
116         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
117         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
a job failure
118         and prunes any partial rows (destroy-AFTER-confirm)."""
119         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
120         ref = storage.parse_uri(uri)
121         n = 0
122         with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
123             local = storage.fetch(
124                 uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
125             )
126             with open(local, newline="", encoding="utf-8") as f:
127                 for row in csv.DictReader(f):
128                     if n >= max_rows:
129                         logger.warning(

```

20. user (2026-07-08T05:45:54.827Z)

```

500         if not _cloud_available(cfg):
501             return _failed(

```

```

502         "cloud-serving was requested but this server isn't configured for it "
503         "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
504         False,
505     )
506     compute = get_compute_backend(cfg, target_override="cloud_run")
507 else:
508     compute = get_compute_backend(
509         cfg
510     ) # server default (default-OFF unless cloud_run)
511 if compute is None:
512     return None # default-OFF ? run the in-process segmenter below (unchanged)
513
514 storage_cfg = getattr(cfg, "storage", None)
515
516 # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
517 try:
518     input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
519 except ValueError as e:
520     return _failed(
521         f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
522     )
523 if input_ref.backend == "local":
524     # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
525     # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
526     # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
527     # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
528     # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
529     # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
530     # default, while a bare/local-looking root can't stage no matter what
input_backend says
531     # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
532     # gives us nothing to build the staged URI under).
533     input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
534     try:
535         root_backend = (
536             storage.parse_uri(input_root).backend if input_root else "local"
537         )
538     except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
539         root_backend = "local"
540     if root_backend == "local":
541         return _failed(
542             "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
543             "stage local uploads into; a path local to this machine can't be read by
the Cloud "
544             "Run job.",
545             False,
546         )
547     local_src = input_ref.key # resolved local path of the upload
548     staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
549     try:
550         notify("staging upload to bucket")
551         # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/

```

```

552         # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
553         storage.put(
554             local_src,
555             staged_uri,
556             config=(
557                 storage_cfg.model_dump() if storage_cfg is not None else None
558             ),
559         )
560     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
561         logger.error(
562             "[API] cloud-serving: failed to stage %s -> %s: %s",
563             local_src,
564             staged_uri,
565             e,
566             exc_info=True,
567         )
568         return _failed(
569             f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
570             False,
571         )
572     # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
573     # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
574     # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
575     req.video_path = staged_uri
576     input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
577     db.add_video(
578         video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
579     )
580     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
581     if not out_root:
582         return _failed(
583             "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
584             "? that's where the Cloud Run job writes the result.",
585             False,
586         )
587
588     notify("dispatching (cloud_run)")
589     logger.info(
590         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
591         req.video_path,
592         out_root,
593     )
594     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
595     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
596     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
597     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
598     overrides = []
599     idx_cfg = getattr(cfg, "indexing", None)
600     sk = getattr(idx_cfg, "skip_ai_handoff", None)
601     if sk is not None:
602         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
603     # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
604     # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving

```

```

preset itself ?
605         # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
606         # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
607         # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
608         # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
609         wasb_cfg = getattr(idx_cfg, "wasb", None)

```

21. user (2026-07-08T05:46:01.615Z)

```

700         "results": [r.model_dump() for r in val_results],
701         "play_segments": segments,
702         "compute": provenance,
703     },
704     True,
705 )
706
707
708 def _execute_processing(
709     config=None, db=None, req=None, progress=None
710 ) -> Dict[str, Any]:
711     """The full hash ? validate ? segment ? report pipeline for one video.
712
713     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
714     module-level ``global_config``/``db_instance`` (backward-compatible with existing
715     tests that pass only ``req``). Route handlers always pass explicit params; older
716     test code that calls ``_execute_processing(req)`` still works.
717
718     ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
719     Raises on unexpected internal errors ? the caller records those as a job failure
720     """
721     import backend.main as _main # call-time seam: main's globals/model stay patchable
722
723     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
724     # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
725     if config is not None and not isinstance(config, AppConfig):
726         # Old signature: _execute_processing(req, progress=None)
727         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
728             req = db
729             db = None
730         elif req is None and isinstance(config, _main.ProcessRequest):
731             req = config
732             config = None
733     # Resolve config/db from module globals when not passed explicitly
734     if config is None:
735         config = _main.global_config
736     if db is None:
737         db = _main.db_instance
738
739     notify = progress or (lambda stage: None)
740
741     notify("hashing")
742     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
743     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
744     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
745     # consumers (hash / validators / segmenter) use the resolved local path.
746     local_video = ""

```

```

747     try:
748         local_video = storage.acquire_local_input(
749             req.video_path, getattr(config, "storage", None)
750         )
751         video_id = _main.compute_video_id(local_video)
752     except Exception:
753         storage.cleanup_acquired_local_input(
754             req.video_path, local_video, getattr(config, "storage", None)
755         )
756         raise
757     try:
758         existing_video = db.get_video(video_id)
759     except Exception:
760         storage.cleanup_acquired_local_input(
761             req.video_path, local_video, getattr(config, "storage", None)
762         )
763         raise
764     was_reingest = existing_video is not None
765
766     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
767     default_seg = getattr(
768         getattr(config, "indexing", None), "default_segmenter", "motion"
769     )
770     seg_name = req.segmenter or default_seg
771     segmenter_actual = None
772     segmentation_ran = False
773     val_results: List[Any] = []
774     val_context: Dict[str, Any] = {}
775     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
776     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
777     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
778     compute_actual: Optional[str] = None
779     response: Optional[Dict[str, Any]] = None
780     try:
781         if (
782             existing_video
783             and existing_video.get("status") == "processed"
784             and not req.force
785         ):
786             cached_segments = db.query_play_segments(video_id, {})
787             if cached_segments:
788                 logger.info(
789                     "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
790                     video_id,
791                 )
792             response = _cached_process_payload(
793                 video_id,
794                 existing_video.get("validation_results", []),
795                 cached_segments,
796                 getattr(req, "compute", None),
797             )
798             return response
799
800     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
801     #     UNLESS this exact content already has a cached verdict under the current
validator config.
802     notify("validating")
803     # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
804     # reupload keys the same row. When that content was ALREADY validated under the

```

```

SAME validator
805         # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
806         # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
807         # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
808         # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
809         # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
810         # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
811         # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
812         # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
813         # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
814         val_fingerprint = registry.config_fingerprint(config)
815         # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
816         # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
817         # failing the job. The cache is a pure speedup ? never a new failure mode.
818         reused_validation = False
819         if not req.force:

```

22. user (2026-07-08T05:46:08.133Z)

```

960         # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
961         if isinstance(response, dict):
962             response.setdefault(
963                 "compute",
964                 {
965                     "path": compute_actual or "not_run",
966                     "requested": getattr(req, "compute", None),
967                 },
968             )
969         prov = response.get("compute", {}) if isinstance(response, dict) else {}
970         logger.info(
971             "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
972             video_id,
973             getattr(req, "compute", None),
974             prov.get("path"),
975             prov.get("dispatched"),
976             (response or {}).get("status") if isinstance(response, dict) else None,
977         )
978         # Always emit the per-video observability report (next to the video, or to
979         # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
980         # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
981         # doesn't surface arbitrary run signals.)
982         paths = _main.emit_indexer_report(
983             db=db,
984             video_id=video_id,
985             # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
986             # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
987             # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
988             # still preserved in DB provenance via db.add_video.

```

```

989         video_path=local_video,
990         config=_report_config_for_input(config, req.video_path),
991         val_results=val_results,
992         val_context=val_context,
993         segmenter_requested=req.segmenter,
994         segmenter_actual=segmenter_actual,
995         was_reingest=was_reingest,
996         segmentation_ran=segmentation_ran,
997     )
998     if paths and isinstance(response, dict):
999         response["reports"] = paths
1000     storage.cleanup_acquired_local_input(
1001         req.video_path, local_video, getattr(config, "storage", None)
1002     )
1003
1004
1005     def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
1006         """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
1007
1008         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
1009         that needs a
1010         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
1011         guard degrades
1012         to a no-op rather than blocking a job it cannot measure."""
1013         try:
1014             sc = (
1015                 storage_config.model_dump()
1016                 if hasattr(storage_config, "model_dump")
1017                 else (storage_config or {})
1018             )
1019             backend = storage.get_storage(input_ref.backend, config=sc)
1020             return int(backend.stat(input_ref).size)
1021         except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
1022             not block
1023             logger.debug(
1024                 "free-space: could not stat remote source %s: %s", input_ref.uri, e
1025             )
1026             return None
1027
1028
1029     class _CompileError(Exception):
1030         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
1031         the API path
1032         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
1033         result."""

```

23. user (2026-07-08T05:46:09.028Z)

```

1130     notify("resolving")
1131     try:
1132         video, kept, out_name = _resolve_compile(
1133             db, cfg, video_id, segment_ids, params.get("output_filename", "")
1134         )
1135     except _CompileError as e:
1136         return {"status": "failed", "message": e.detail, "video_id": video_id}
1137
1138     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1139     output_dir = (
1140         getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1141     )
1142     # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
1143     the job);
1144     # unknown/absent locale keeps the configured default unchanged.
1145     localized_watermark = localize_watermark(stitching.watermark, locale)

```

```

1145     compiler = _main.VideoCompiler(
1146         output_dir,
1147         begin_padding=stitching.begin_padding,
1148         end_padding=stitching.end_padding,
1149         watermark=localized_watermark,
1150     )
1151     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1152
1153     notify("stitching")
1154     local_src = ""
1155     try:
1156         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
1157         # path). Resolve to
1158         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
1159         # scratch for a URI (the
1160         # same seam the process path uses); without it, compiling a bucket-ingested
1161         # video can't open gs://.
1162         local_src = storage.acquire_local_input(
1163             video["filepath"], getattr(cfg, "storage", None)
1164         )
1165         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
1166         # (for remote
1167         # dispatch) doesn't run multiple compiles at once on the box (see
1168         # _LOCAL_COMPUTE_LANE).
1169         with _LOCAL_COMPUTE_LANE:
1170             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1171         except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
1172             FAILURE, not a crash
1173             logger.error(
1174                 "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1175             )
1176         return {
1177             "status": "failed",
1178             "message": f"Highlight stitching failed: {e}",
1179             "video_id": video_id,
1180         }
1181     finally:
1182         if local_src:
1183             storage.cleanup_acquired_local_input(
1184                 video["filepath"], local_src, getattr(cfg, "storage", None)
1185             )
1186
1187     # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
1188     # URL ? Cloud Run
1189     # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
1190     # copy dies when the
1191     # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
1192     # (the endpoint
1193     # still falls back to the local file on the same instance).
1194     reel_uri = reel_object_uri(cfg, video_id, out_name)
1195     if reel_uri:
1196         try:
1197             notify("uploading reel")
1198             # Thread the active storage settings so the configured backend (S3
1199             # endpoint/region, GCS
1200             # project/client) is used, not a fresh default.
1201             storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1202         except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
1203             already succeeded
1204             logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1205
1206     # `filepath` is server-local (not browser-reachable); also return the download URL
1207     # so the SPA needn't
1208     # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1209     download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"

```

```

1198     notify("finished")
1199     return {
1200         "status": "success",
1201         "filepath": out_path,
1202         "download_url": download_url,
1203         "video_id": video_id,
1204     }

```

24. user (2026-07-08T05:46:17.783Z)

```

620     fetched once into an output-scoped source cache, then served and proxied from that
stable copy. """
621     rec = request.app.state.db.get_video(video_id)
622     if not rec or not rec.get("filepath"):
623         raise HTTPException(status_code=404, detail="Video not found.")
624     cfg = request.app.state.config
625
626     # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
627     try:
628         proxy_name = safe_output_filename(f"{video_id}.mp4")
629         proxy_path = resolve_under_root(
630             os.path.join(_proxies_dir(cfg), proxy_name), _proxies_dir(cfg)
631         )
632     except ValueError:
633         proxy_path = ""
634     if proxy_path and os.path.isfile(proxy_path):
635         return FileResponse(proxy_path, media_type="video/mp4")
636
637     # 2) Otherwise resolve + serve the source.
638     try:
639         input_ref = storage.resolve_input_ref(
640             rec["filepath"], getattr(cfg, "storage", None)
641         )
642     except (
643         ValueError
644     ) as e: # unsupported scheme etc. ? a clean client error, never a 500
645         raise HTTPException(status_code=400, detail=str(e)) from e
646     except Exception as e: # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced
(not swallowed)
647         raise HTTPException(
648             status_code=502, detail=f"Could not resolve the source video: {e}"
649         ) from e
650
651     if input_ref.backend == "local":
652         local = input_ref.key
653     else:
654         try:
655             local = _cached_remote_source(video_id, input_ref, cfg)
656         except HTTPException:
657             raise
658     except Exception as e: # noqa: BLE001 - a fetch miss is surfaced (not
swallowed)
659         raise HTTPException(
660             status_code=502, detail=f"Could not resolve the source video: {e}"
661         ) from e
662     if not os.path.isfile(local):
663         raise HTTPException(status_code=404, detail="Source video file not found.")
664
665     # 3) Warm a proxy in the background for next time.
666     threading.Thread(
667         target=_warm_annotation_proxy, args=(video_id, local, cfg), daemon=True
668     ).start()
669
670     media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
671     return FileResponse(local, media_type=media)

```

```

672
673
674     @app.post("/api/annotations")
675     def post_annotations(req: AnnotationsRequest, request: Request):
676         """Persist finished rally labels as the annotator CSV (P8), written verbatim to
677         ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5
678         ``video_id`` ? so it joins
679         the video by content hash exactly as the training pipeline expects. Byte-compatible
680         with the VLC
681         plugin + browser extension. A later step (owner-gated) promotes labels into the
682         durable corpus.

```

25. user (2026-07-08T05:46:18.687Z)

```

770         if bool(req.uploaded_path) == bool(req.source_uri):
771             raise HTTPException(
772                 status_code=400,
773                 detail="provide exactly one of 'uploaded_path' or 'source_uri'.",
774             )
775         if req.uploaded_path and not req.medium_uri:
776             raise HTTPException(
777                 status_code=400, detail="'medium_uri' is required when uploading a file."
778             )
779
780         storage_settings = _storage_settings(cfg)
781
782         # 1) Resolve a LOCAL working copy of the source + its whole-file MD5 (the join key,
783         D3).
784         if req.uploaded_path:
785             try:
786                 validate_input_path(req.uploaded_path, allowed_root=None)
787                 src_ref = storage.resolve_input_ref(req.uploaded_path, storage_settings)
788             except ValueError as e:
789                 raise HTTPException(status_code=400, detail=str(e)) from e
790             if src_ref.backend != "local" or not os.path.isfile(src_ref.key):
791                 raise HTTPException(
792                     status_code=404,
793                     detail=f"uploaded file not found at '{req.uploaded_path}'",
794                 )
795             local_working = src_ref.key
796         else:
797             try:
798                 validate_input_path(req.source_uri, allowed_root=None)
799                 src_ref = storage.resolve_input_ref(req.source_uri, storage_settings)
800             except ValueError as e:
801                 raise HTTPException(status_code=400, detail=str(e)) from e
802             # A remote at-rest source is fetched to scratch to hash it (D3) ? reserve that
803             scratch (P2).
804             if src_ref.backend != "local":
805                 need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage", None))
806                 require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
807             try:
808                 local_working = storage.acquire_local_input(
809                     req.source_uri, getattr(cfg, "storage", None)
810                 )
811             except ValueError as e:
812                 raise HTTPException(status_code=400, detail=str(e)) from e
813             except Exception as e: # noqa: BLE001 ? a fetch miss is a 502, surfaced not
814                 swallowed
815                 raise HTTPException(
816                     status_code=502, detail=f"could not resolve source_uri: {e}"
817                 ) from e
818             if not os.path.isfile(local_working):
819                 raise HTTPException(
820                     status_code=404, detail="source_uri did not resolve to a readable file."

```

```

818         )
819
820     cleanup_raw = req.source_uri or ""
821     try:
822         video_id = compute_video_id(local_working)
823         ext = os.path.splitext(local_working)[1] or ".mp4"
824
825         # 2) Decide the durable location + copy only when needed.
826         copied = False
827         if req.source_uri:
828             stored_uri = (
829                 req.source_uri
830             ) # already at rest in its medium ? no copy (D14b)
831             medium_backend = src_ref.backend
832         else:
833             try:
834                 medium_ref = storage.parse_uri(req.medium_uri)
835             except ValueError as e:
836                 raise HTTPException(
837                     status_code=400, detail=f"invalid medium_uri: {e}"
838                 ) from e
839             medium_backend = medium_ref.backend
840             if medium_ref.backend == "local":
841                 # "This computer" ? the upload already lives here; it IS the durable
842                 copy (no duplicate).
843                 stored_uri = os.path.abspath(local_working)
844             else:
845                 dest = _asset_dest_uri(req.medium_uri, video_id, ext)
846                 # Free-space: BLOCK a local/Drive-mount destination (D11); a cloud
847                 bucket is warn-only.
848                 local_dir = _medium_local_dir(medium_ref, storage_settings)
849                 if local_dir is not None:
850                     require_free_bytes(
851                         local_dir, os.path.getsize(local_working), stage="store"
852                     )
853                 else:
854                     logger.info(
855                         "[ASSETS] cloud medium %s ? capacity not blocked (cost/quota
856                         warn only).",
857                         medium_ref.backend,
858                     )
859                 try:
860                     stored_uri = storage.put(
861                         local_working, dest, config=storage_settings
862                     )
863                     copied = True
864                 except ValueError as e:
865                     raise HTTPException(status_code=400, detail=str(e)) from e
866                 except Exception as e: # noqa: BLE001 ? a put failure is surfaced,
867                     never silently dropped
868                     logger.error(
869                         "[ASSETS] put failed video_id=%s dest=%s: %s",
870                         video_id,
871                         dest,
872                         e,
873                         exc_info=True,
874                     )
875                     raise HTTPException(
876                         status_code=502, detail=f"could not store into the medium: {e}"
877                     ) from e
878
879         # 3) Register the asset (keyed by MD5) so it appears in GET /api/videos ("my
880         videos").
881         if not request.app.state.db.add_video(video_id, stored_uri, "stored", []):
882             raise HTTPException(status_code=500, detail="failed to register the asset.")

```

```

878         logger.info(
879             "[ASSETS] stored video_id=%s medium=%s uri=%s copied=%s owner=%s",
880             video_id,
881             medium_backend,
882             stored_uri,
883             copied,
884             owner_id,
885         )
886     return JsonResponse(
887         status_code=201,
888         content={
889             "video_id": video_id,
890             "medium": medium_backend,
891             "stored_uri": stored_uri,
892             "copied": copied,
893             "sport": req.sport,
894             "status": "stored",
895         },
896     )
897 finally:
898     if cleanup_raw:
899         storage.cleanup_acquired_local_input(
900             cleanup_raw, local_working, getattr(cfg, "storage", None)
901         )
902
903
904 @app.post("/api/process", status_code=202)
905 def process_video(req: ProcessRequest, request: Request):
906     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
907
908     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
909     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
910     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
911     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
912     """
913     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
914     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
915     # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
916     try:
917         owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
918     except edge_auth.EdgeAuthError as e:
919         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
920
921     allowed_root = getattr(
922         getattr(request.app.state.config, "security", None), "allowed_video_root", None
923     )
924     # In hosted mode a bucket URI is allowed only when it's under the configured
input_root (the
925     # designated bucket) ? not an arbitrary bucket the runtime SA happens to reach
(tenant isolation).
926     allowed_uri_root = getattr(
927         getattr(request.app.state.config, "storage", None), "input_root", None
928     )
929     try:
930         validate_input_path(
931             req.video_path, allowed_root=allowed_root, allowed_uri_root=allowed_uri_root
932         )
933     # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
934     # scheme, which we map to a clean 400 (an unknown scheme must never 500).
935     input_ref = storage.resolve_input_ref(
936         req.video_path, getattr(request.app.state.config, "storage", None)

```

```

937         )
938     except ValueError as e:
939         raise HTTPException(status_code=400, detail=str(e)) from e
940     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
941     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
942     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
943     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
944     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
945     # rejects a non-local URI as out-of-root.
946     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
947         raise HTTPException(
948             status_code=404, detail=f"Video file not found at '{req.video_path}'"
949         )

```

26. user (2026-07-08T05:46:27.284Z)

```

backend\api\jobs.py:74:     from backend import orchestration # call-time seam (mirrors
submit_process_job)
backend\main.py:580:         storage.fetch(input_ref.uri, tmp, config=_storage_settings(cfg))
backend\main.py:681:     Edge-auth mirrors /api/process (default-OFF ? no-op locally).
``video_id`` is validated as a bare
backend\main.py:712:def _storage_settings(cfg) -> Dict[str, Any]:
backend\main.py:748:     Edge-auth mirrors ``/api/process``. Exactly one source: an
``uploaded_path`` (a local file from
backend\main.py:780:     storage_settings = _storage_settings(cfg)
backend\main.py:857:         stored_uri = storage.put(
backend\main.py:1052:     # #463: when the reel was mirrored to the output bucket, 307 to a
short-lived signed URL (bytes flow
backend\main.py:1102:     Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each
clip + concat + watermark)
backend\config\models.py:465:     Valid values mirror ``backend/config/presets.py`` (parity-
pinned by a unit test)."""
backend\api\job_worker.py:422:         sc = orchestration._storage_settings(config)
backend\config\presets.py:25:# Canonical enum values, defined ONCE here.
``models.DeploymentConfig``'s ``Literal`` mirrors
backend\flywheel.py:65:     source video. Returns the written URIs. Reuses
:func:`backend.storage.put` + the ``VideoWorkspace``
backend\flywheel.py:78:         written["manifest_uri"] = _storage.put(manifest_local,
manifest_uri, config=cfg)
backend\flywheel.py:83:         written["source_uri"] = _storage.put(source_path, source_uri,
config=cfg)
backend\eval\ablation.py:112:     flagged dead weight on "no measured lift" (mirrors
``per_user_gate`` ``structural=True``).
backend\eval\ablation.py:457:     the loader never crashes, mirroring
``experiment._feature_column``'s tolerance.
backend\infrastructure\database_jobs.py:208:         the owner scoping mirrors ``list_jobs``
(NULL ? local single-user)."""
backend\features\rally_seq.py:55:     occlusion gap (mirrors the heuristic resetting on
invisibility)."""
backend\worker\_main__.py:148:         storage.put(os.path.join(out_local, fn), uri,
config=storage_cfg)
backend\worker\_main__.py:238:         storage.put(local, done_uri, config=storage_cfg)
backend\worker\_main__.py:422:#: Video extensions accepted under <corpus>/videos/ (mirrors the
labels/ '.csv' filter so a
backend\worker\_main__.py:699:         storage.put(os.path.join(bundle, fn), uri,
config=storage_cfg)
backend\orchestration.py:227:     sc = _storage_settings(cfg)
backend\orchestration.py:302:         sc = _storage_settings(cfg)
backend\orchestration.py:328:     branch mirrors the real cloud ingest exactly (watermarks +
``_finalize_reingest``), so a

```

```

backend\orchestration.py:347:         sc = _storage_settings(cfg)
backend\orchestration.py:528:         # built by joining under input_root, and storage.put routes
by parse_uri(staged_uri).backend ?
backend\orchestration.py:553:         storage.put(
backend\orchestration.py:811:         # ``force=true`` bypasses the reuse and re-writes a fresh
verdict below (mirrors how it bypasses
backend\orchestration.py:1085: def reel_object_uri(cfg, video_id: str, name: str) -> str:
backend\orchestration.py:1086:     """The bucket URI a compiled reel is mirrored to for a direct
signed-URL download (#463), or "" when
backend\orchestration.py:1088:     mirror-upload and the /api/highlights redirect:
``<output_root>/highlights/<video_id>/<name>``."""
backend\orchestration.py:1095: def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
backend\orchestration.py:1104:     """A short-lived signed URL for a reel mirrored to the output
bucket, so /api/highlights can
backend\orchestration.py:1108:     reel_uri = reel_object_uri(cfg, video_id, name)
backend\orchestration.py:1111:     settings = _storage_settings(cfg)
backend\orchestration.py:1181:     # #463: mirror the reel to the output bucket so
/api/highlights can 307 to a signed URL ? Cloud Run
backend\orchestration.py:1183:     # instance recycles. A mirror-upload failure must NOT fail an
otherwise-good stitch (the endpoint
backend\orchestration.py:1185:     reel_uri = reel_object_uri(cfg, video_id, out_name)
backend\orchestration.py:1191:     storage.put(out_path, reel_uri,
config=_storage_settings(cfg))
backend\orchestration.py:1192:     except Exception as e: # noqa: BLE001 ? best-effort
mirror; local stitch already succeeded
backend\orchestration.py:1193:         logger.warning("[compile] could not mirror reel to %s:
%s", reel_uri, e)
backend\compute\base.py:56:     """The terminal state of a dispatched job. ``returncode`` mirrors
the worker's exit code
backend\compute\cloud_run.py:152:         # (mirrors run_job's project guard, #342) so the misuse
is exercisable in CI (where
backend\compute\cloud_run.py:342:         # race where the worker wrote it between our two reads
(mirrors the post-timeout rescue in
backend\utils\ffmpeg.py:80:     """The OS-appropriate one-line install command (mirrors
scripts/doctor.py)."""
backend\pipeline\personalization\serving.py:45: # override a baseline fusion default ? we fall
back to the served baseline. This mirrors
backend\eval\experiment.py:66: # the gitignored output/) so the leaderboard survives a clean
checkout, mirroring
backend\eval\experiment.py:235:     ``no_audio_error`` formatted with ``path``),
mirroring "run ``--augment`` first".
backend\utils\workspace.py:14: keeping, and a **fast local compute-scratch** mirror (always
local) for frame caches / extracted
backend\utils\workspace.py:113:     """The fast LOCAL compute-scratch mirror for a video (always
native FS, never a cloud root).
backend\pipeline\detectors\fusion_features.py:3: NO I/O, no torch, no cv2 ? mirrors
`rally_gate.py`'s discipline so the whole feature
backend\eval\golden_fixtures.py:642:     Key-set equality is enforced on BOTH the top-level dict
AND the ``score`` block (mirroring the
backend\pipeline\stitcher\compiler.py:71:         This mirrors the parse_time logic used in
gemini.py to ensure
backend\eval\per_user_eval.py:20: ``len(videos) - k`` are *held-out* (scored, never seen during
fit). This mirrors the product
backend\eval\per_user_regression.py:8: This module builds that missing per-user regression
baseline, mirroring the shared discipline.
backend\eval\per_user_regression.py:122:     incumbent is detectable rather than silently trusted
(mirrors regression.py's
backend\eval\gemini_refine.py:14: This mirrors wasb_hybrid's Phase-3 handoff but runs standalone
on a cached
backend\eval\gemini_refine.py:104:         # isn't mistaken for a complete one (mirrors the
gemini segmenter / hybrids).
backend\eval\gemini_refine.py:264:         # pass isn't mistaken for a complete one (mirrors
the gemini segmenter / hybrids).
backend\eval\rally_seg_eval.py:225:     NEVER-EMPTY safeguard (mirrors
``FusionSegmenter._select_for_handoff``): if the gate would

```

```

backend\eval\served_gate_a.py:45:    """The TYPED ``indexing`` config the served FusionSegmenter
seams read ? mirrors config.json's
backend\storage\gdrive_api.py:12:fully-offline tests (mirroring ``GCSStorage(client=...)``).
backend\pipeline\detectors\wasb_infer.py:317:    It mirrors the on-disk names ``extract_frames``
writes (``{i:08d}.png``) so the
backend\pipeline\detectors\wasb_infer.py:1117:        # ---- cache setup / resume decision
(mirrors run()); keyed nvdec_resident) --- #
backend\pipeline\segmenters\yolo_hybrid.py:72:        mirrors the gemini/trajectory segmenters).
A genuine "ran, found nothing" is unaffected.
backend\pipeline\segmenters\yolo_hybrid.py:368:        # partial run isn't mistaken for a
complete one (mirrors the gemini segmenter).
backend\pipeline\segmenters\trajectory_hybrid.py:195:        # Unrunnable config, not a
0-rally match (mirrors the gemini segmenter).
backend\pipeline\segmenters\trajectory_hybrid.py:223:        # marked failed instead of
a silent empty "success" (mirrors gemini; the worker's

```

[Showing results with pagination = limit: 70]

27. user (2026-07-08T05:46:28.769Z)

```

backend\main.py:580:        storage.fetch(input_ref.uri, tmp, config=_storage_settings(cfg))
backend\orchestration.py:123:        local = storage.fetch(
backend\orchestration.py:266:        local = storage.fetch(
backend\worker\__main__.py:265:        local_video = storage.fetch(
backend\worker\__main__.py:594:        local_label = storage.fetch(
backend\worker\__main__.py:604:        local_video = storage.fetch(
backend\worker\__main__.py:680:        storage.fetch(

```

28. user (2026-07-08T05:46:36.005Z)

```

1      """Per-video workspace path helper ? one folder per video, local?cloud identical.
2
3      See ``docs/PER_VIDEO_WORKSPACE.md``. The goal: every per-video artifact lives under
4      ONE folder keyed by the MD5 ``video_id``
5      (``backend/utils/hashing.py::compute_video_id``):
6
7      <root>/[<prefix>]/<video_id>/{source,detector,outputs,reels,logs,tmp}/...
8
9      The ``root`` is a **storage URI** (``local`` path / ``gcs://`` / ``s3://`` /
10     ``gdrive://``), so a
11     LOCAL deployment and a CLOUD (bucket) deployment share the *same relative layout* ? only
12     the root
13     differs. This module is **pure** (string/URI joins, no I/O) so it is trivially testable
14     and works
15     for every backend; the storage layer (``backend/storage/``) does the actual read/write.
16
17     Two roots by design (D3): a **durable** workspace root (may be ``gcs://``) for artifacts
18     worth
19     keeping, and a **fast local compute-scratch** mirror (always local) for frame caches /
20     extracted
21     chunks that must never be written to a cloud root. ``tmp/`` is the local-only bucket.
22
23     This is Phase 0: the helper + config fields only ? NOT yet wired into write sites (that
24     converges
25     in P2?P4, flag-gated by ``storage.single_folder_per_video``). No behaviour changes on
26     import.
27
28     """
29
30     from __future__ import annotations
31
32     import os
33     from dataclasses import dataclass
34
35     # Per-video subfolders. ``tmp`` is LOCAL-ONLY (frame PNGs, handoff/yolo/gemini chunks,
36     compile

```

```

27     # clips) ? huge + ephemeral, never pushed to a cloud workspace; cache-hygiene drops it.
28     SUBDIRS = ("source", "detector", "outputs", "reels", "logs", "tmp")
29     LOCAL_ONLY_SUBDIRS = ("tmp",)
30
31
32     def join_uri(root: str, *parts: str) -> str:
33         """Join path components onto a storage root with forward slashes.
34
35         Matches how the codebase already composes storage URIs (e.g.
36         ``f"{prefix}/weights/..."`` in ``backend/worker``). Works for ``gcs://bucket``, ``s3://bucket``, ``local:///abs``
37         and bare paths alike; empty/blank parts are skipped so ``join_uri(root, "", "x")`` ==
38         ``join_uri(root, "x")``. Forward slashes are valid path separators in Python on Windows too.
39         """
40         base = root.rstrip("/")
41         # Skip empty / whitespace-only parts; for real parts strip only slashes so spaces
42         # inside a # name (e.g. a mounted "My Drive" path) are preserved.
43         tail = "/".join(p.strip("/") for p in parts if p and p.strip())
44         return f"{base}/{tail}" if tail else base
45
46
47     def resolve_workspace_root(
48         workspace_root: str, output_root: str, output_dir: str
49     ) -> str:
50         """Durable workspace root, by precedence: ``storage.workspace_root`` ?
51         ``storage.output_root`` ? ``output.output_dir``. The first non-empty wins (non-
52         breaking)."""
53         return (
54             (workspace_root or "").strip()
55             or (output_root or "").strip()
56             or (output_dir or "output")
57         )
58
59     @dataclass(frozen=True)
60     class VideoWorkspace:
61         """The single folder for one video. Pure path math; build with :func:`for_video`.
62
63         ``base`` = ``<root>/[<prefix>]/<video_id>``. Subfolder accessors return URIs/paths
64         under it.
65
66         """
67         root: str
68         video_id: str
69         prefix: str = "workspaces"
70
71         @property
72         def base(self) -> str:
73             if self.prefix:
74                 return join_uri(self.root, self.prefix, self.video_id)
75             return join_uri(self.root, self.video_id)
76
77         def path(self, sub: str = "", name: str = "") -> str:
78             """A path under this video's folder: ``base[/sub][/name]``."""
79             return join_uri(self.base, sub, name)
80
81         @property
82         def source(self) -> str:
83             return self.path("source")
84
85         @property
86         def detector(self) -> str:

```

```

86         return self.path("detector")
87
88     @property
89     def outputs(self) -> str:
90         return self.path("outputs")
91
92     @property
93     def reels(self) -> str:
94         return self.path("reels")
95
96     @property
97     def logs(self) -> str:
98         return self.path("logs")
99
100    @property
101    def tmp(self) -> str:
102        return self.path("tmp")
103
104
105    def for_video(root: str, video_id: str, prefix: str = "workspaces") -> VideoWorkspace:
106        """Build the :class:`VideoWorkspace` for ``video_id`` under ``root`` (a storage
107        URI)."""
108        if not video_id:
109            raise ValueError("video_id is required to build a per-video workspace")
110        return VideoWorkspace(root=root, video_id=video_id, prefix=prefix)
111
112    def local_scratch_base(scratch_root: str, video_id: str) -> str:
113        """The fast LOCAL compute-scratch mirror for a video (always native FS, never a
114        cloud root).
115
116        Used for frame caches / extracted chunks (the ``tmp/`` bucket) during compute,
117        before durable
118        artifacts are pushed to the workspace. Uses ``os.path.join`` (OS-native separators)
119        since this
120        is always a local filesystem path.
121        """
122        if not video_id:
123            raise ValueError("video_id is required for a local scratch path")
124        return os.path.join(scratch_root or os.getcwd(), video_id, "tmp")

```

29. user (2026-07-08T05:46:37.194Z)

```

120    video_id: str,
121    segments: List[dict],
122    status: str,
123    storage_cfg: dict,
124    video_uri: str = "",
125    ) -> List[str]:
126        """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
127        push both to
128        ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
129
130        Shared by the SUCCESS path and the failure path (``_fail``) so a detector
131        timeout/abort leaves a
132        ``status="failed"`` report.json in the bucket ? the control plane's source of truth
133        (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
134        non-``success``
135        report as "no usable result" ? retry, and the restart-recovery poll waits for
136        report.json to
137        EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``.
138        """
139        out_local = os.path.join(scratch, "outputs", video_id)
140        os.makedirs(out_local, exist_ok=True)
141        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))

```

```

137     _write_report_json(
138         video_id,
139         segments,
140         status,
141         os.path.join(out_local, "report.json"),
142         video_uri=video_uri,
143     )
144     out_prefix = out_uri.rstrip("/")
145     pushed: List[str] = []
146     for fn in sorted(os.listdir(out_local)):
147         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149         pushed.append(uri)
150     return pushed
151
152
153 def _run_startup_checks(config: Any) -> bool:
154     """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155     best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156     cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157     (returns True, ZERO work) when no deployment.profile is selected.
158
159     The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160     torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161     WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162     no-op of work, not just of output."""
163     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164     cuda = (
165         probe_cuda_available() if profile else None
166     ) # torch-free on the default-OFF path
167     try:
168         enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169         return True
170     except StartupCheckError:
171         return False # already logged at ERROR by enforce_startup_checks
172
173
174 def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175     """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176     outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177     container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179     A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180     from the local 2/3), never a raw traceback:
181     * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution
may still be
182     running (the shim best-effort cancels it).
183     * rc 5 ? the execution has TERMINATED without a marker
(``RemoteExecutionFailed``): a worker
184     crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
185     ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
186     from backend.compute import RemoteExecutionFailed
187     from backend.infrastructure.execution_policy import PolicyTimeout
188

```

```

189     spec = ComputeJobSpec(
190         video_uri=args.video_uri,
191         out_uri=args.out_uri,
192         segmenter=args.segmenter,
193         config_overrides=tuple(args.set or ()),
194         skip_validation=bool(getattr(args, "skip_validation", False)),
195     )
196     try:
197         result = compute.run_infer(spec)
198     except RemoteExecutionFailed as e:
199         logger.error(
200             "[worker] remote execution terminated without a completion marker: %s", e
201         )
202         return 5
203     except PolicyTimeout as e:
204         logger.warning(
205             "[worker] remote compute did not complete (no marker within budget): %s", e
206         )
207         return 4
208     logger.info(
209         "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210         result.video_id,
211         result.returncode,
212         result.outputs_uri,
213     )
214     return result.returncode
215
216
217 def _write_done_marker(
218     done_uri: str,
219     video_id: str,
220     returncode: int,
221     segment_count: int,
222     status: str,
223     storage_cfg: dict,
224     scratch: str,
225 ) -> None:
226     """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
227     for a remote
228     dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
229     return."""
230     try:
231         marker = {
232             "video_id": video_id,
233             "returncode": returncode,
234             "segment_count": segment_count,
235             "status": status,
236         }
237         local = os.path.join(scratch, "_done.json")
238         with open(local, "w", encoding="utf-8") as f:
239             json.dump(marker, f)
240         storage.put(local, done_uri, config=storage_cfg)
241         logger.info("[worker] wrote completion marker -> %s", done_uri)
242     except Exception as e: # never fail a good run because the marker push hiccuped
243         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
244
245
246 def cmd_infer(args: argparse.Namespace) -> int:
247     config = load_config(args.config) if args.config else load_config()
248     _apply_overrides(config, args.set)
249     if not _run_startup_checks(config):
250         return 2
251
252     # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
253     process.

```

```

251      # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
252      # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
253      # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
254      compute = get_compute_backend(config)
255      if compute is not None:
256          return _dispatch_remote(compute, args)
257
258      storage_cfg = config.storage.model_dump()
259
260      scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
261      os.makedirs(scratch, exist_ok=True)
262      config.output.output_dir = scratch
263
264      # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
265      local_video = storage.fetch(
266          args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
267      )
268      print(f"[worker] pulled {args.video_uri} -> {local_video}")
269
270      # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
271      video_id = compute_video_id(local_video)
272
273      # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274      # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
275      # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276      # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
277      # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278      # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279      # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.

```

30. user (2026-07-08T05:46:48.732Z)

```

545      def _source_cache_dir(cfg) -> str:
546          return os.path.join(output_base(cfg), "source_cache")
547
548
549      def _source_cache_lock(video_id: str) -> threading.Lock:
550          with _source_cache_locks_guard:
551              return _source_cache_locks.setdefault(video_id, threading.Lock())
552
553
554      def _source_cache_path(video_id: str, input_ref, cfg) -> str:
555          ext = os.path.splitext(input_ref.name or "")[1].lower()
556          if not ext or "/" in ext or "\\" in ext or len(ext) > 16:
557              ext = ".mp4"
558          cache_dir = _source_cache_dir(cfg)
559          name = safe_output_filename(f"{video_id}{ext}")
560          return resolve_under_root(os.path.join(cache_dir, name), cache_dir)
561
562
563      def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
564          """Fetch a remote source once into an output-scoped source/proxy cache."""
565          cache_path = _source_cache_path(video_id, input_ref, cfg)
566          if os.path.isfile(cache_path):

```

```

567         return cache_path
568     lock = _source_cache_lock(video_id)
569     with lock:
570         if os.path.isfile(cache_path):
571             return cache_path
572         cache_dir = os.path.dirname(cache_path)
573         os.makedirs(cache_dir, exist_ok=True)
574         need = _remote_input_size_bytes(input_ref, getattr(cfg, "storage", None))
575         require_free_bytes(cache_dir, need, stage="source-cache")
576         tmp = os.path.join(
577             cache_dir, f".{os.path.basename(cache_path)}.{uuid.uuid4().hex}.tmp"
578         )
579         try:
580             storage.fetch(input_ref.uri, tmp, config=_storage_settings(cfg))
581             os.replace(tmp, cache_path)
582         finally:
583             try:
584                 if os.path.exists(tmp):
585                     os.remove(tmp)
586             except OSError:
587                 pass
588         return cache_path
589
590
591     def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
592         """Background, best-effort: generate the annotation proxy ONCE so subsequent
593         /api/source loads
594         serve the lighter, seekable copy. Deduped; never blocks the request; the source
595         keeps serving if
596         it fails."""
597         with _proxy_lock:
598             if video_id in _proxy_inflight:
599                 return
600             _proxy_inflight.add(video_id)
601         try:
602             from backend.utils.annotation_proxy import make_annotation_proxy, needs_proxy
603
604             if not needs_proxy(src_local):
605                 return # already light enough ? no proxy worth making
606             make_annotation_proxy(
607                 src_local, os.path.join(_proxies_dir(cfg), f"{video_id}.mp4")
608             )
609         except Exception as e: # noqa: BLE001 - warming is best-effort
610             logger.warning("proxy: warm failed for %s: %s", video_id, e)
611         finally:
612             with _proxy_lock:
613                 _proxy_inflight.discard(video_id)
614
615     @app.get("/api/source/{video_id}")
616     def get_source(video_id: str, request: Request):
617         """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub.
618         Serves a
619         frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels
620         still keyed
621         to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy
622         in the
623         background for next time ? non-blocking, so the first load is never delayed. Remote
624         sources are

```

31. user (2026-07-08T05:46:49.674Z)

```

705     )
706     logger.info(
707         "annotations saved: video_id=%s rows=%d -> %s", req.video_id, num_rows, path

```

```

708         )
709         return {"status": "ok", "num_rows": num_rows, "csv_uri": f"labels/{name}"}
710
711
712     def _storage_settings(cfg) -> Dict[str, Any]:
713         """The storage config as a plain settings dict (NO secrets ? see StorageConfig)."""
714         sc = getattr(cfg, "storage", None)
715         if sc is None:
716             return {}
717         return sc.model_dump() if hasattr(sc, "model_dump") else dict(sc)
718
719
720     def _asset_dest_uri(medium_uri: str, video_id: str, ext: str) -> str:
721         """The durable, MD5-keyed destination inside the chosen medium:
722         ``<medium>/<video_id><ext>``. """
723         return f"{medium_uri.rstrip('/')}/{video_id}{ext}"
724
725
726     def _medium_local_dir(medium_ref, storage_settings: Dict[str, Any]) -> Optional[str]:
727         """The LOCAL directory a ``put`` into this medium would write into (so a free-space
728         BLOCK can be enforced, D11), or ``None`` for a cloud bucket (which is cost/quota-warned, never
729         blocked).
730         ``local`` ? the folder; ``gdrive`` ? the resolved path under the Drive mount. """
731         if medium_ref.backend == "local":
732             return medium_ref.key or "."
733         if medium_ref.backend == "gdrive":
734             from backend.storage.capabilities import _gdrive_mount_root
735
736             root = _gdrive_mount_root(storage_settings)
737             if not root:
738                 return None
739             rel = medium_ref.key.lstrip("/")
740             return os.path.join(root, *rel.split("/")) if rel else root
741         return None # s3 / gcs ? a bucket has no readable local capacity
742
743
744     @app.post("/api/assets", status_code=201)
745     def create_asset(req: AssetRequest, request: Request):
746         """Store a video into the user's chosen medium and register it as a library asset
747         (P4, D14).
748
749         SELF-HOST posture only (D7): refused with 403 when ``security.allowed_video_root``
750         is set ? the
751         hosted alpha jails inputs and has no per-user medium credentials, so it stays
752         upload-to-local.
753         Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a
754         local file from
755         the chunked upload) OR a ``source_uri`` (a video already at rest in a medium). An
756         uploaded file +

```

32. user (2026-07-08T05:46:50.288Z)

```

1080         f"stitching.min_shot_count={stitching.min_shot_count}.",
1081         )
1082         return video, kept, out_name
1083
1084
1085     def reel_object_uri(cfg, video_id: str, name: str) -> str:
1086         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
1087         (#463), or "" when
1088         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
1089         compile
1090         mirror-upload and the /api/highlights redirect:
1091         ``<output_root>/highlights/<video_id>/<name>``. """

```

```

1089     root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
1090     if not root or "://" not in root:
1091         return ""
1092     return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1093
1094
1095     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1096         """The active storage settings dict (per-backend config for the storage layer ? S3
1097         endpoint/region,
1098         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
1099         default backend
1100         isn't silently constructed as a fresh default."""
1101         sc = getattr(cfg, "storage", None)
1102         return sc.model_dump() if sc is not None else None
1103
1104     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1105         """A short-lived signed URL for a reel mirrored to the output bucket, so
1106         /api/highlights can
1107         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
1108         ephemeral local
1109         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
1110         config for BOTH
1111         the existence check and the signing (extracted here to keep backend/main.py lean ?
1112         P23/P24)."""
1113         reel_uri = reel_object_uri(cfg, video_id, name)
1114         if not reel_uri:
1115             return None
1116         settings = _storage_settings(cfg)
1117         if storage.exists(reel_uri, config=settings):
1118             return storage.signed_url(reel_uri, config=settings, method="GET")
1119         return None
1120
1121     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1122         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
1123         stitch the reel,
1124         and return the {status, filepath, download_url} result the SPA polls for. Returns a
1125         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
1126         stitch error)
1127         so the SPA always gets a clean verdict instead of a worker-crash error."""
1128         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
1129         backend.main
1130
1131         notify = progress or (lambda stage: None)
1132         params = json.loads(job.get("params") or "{}")
1133         video_id = job.get("video_id") or ""
1134         segment_ids = params.get("segment_ids") or []
1135         locale = params.get("locale")

```

33. user (2026-07-08T05:46:59.046Z)

```

40         return storage_config.model_dump()
41     return storage_config if isinstance(storage_config, dict) else None
42
43
44     def consent_attested(job: Dict[str, Any], config: Any = None) -> bool:
45         """The flywheel's consent gate ? currently a **FAKED hard-deny** (always ``False``).
46
47         The flywheel may capture a user's footage into the training pool ONLY with attested,
48         **adults-only**
49         consent (D10 / the CLAUDE.md guardrail: minors excluded; per-user training is a
50         separate explicit
51         opt-in; paid tier never trains). The REAL gate ? reading counsel-defined consent

```

```

columns + an
50         adults-only attestation + a retention class ? lands with **M9-consent**; until then
this denies
51         every job so the harness captures nothing. **Do not relax without counsel sign-
off.**"""
52         return False
53
54
55     def capture_job(
56         workspace: "_workspace.VideoWorkspace",
57         *,
58         rally_output: Dict[str, Any],
59         source_path: Optional[str] = None,
60         storage_config: Any = None,
61     ) -> Dict[str, str]:
62         """Durably persist a consented job's artifacts into its per-video ``workspace`` (the
raw material a
63         human later labels ? the golden training set). Always writes a small
``capture.json`` manifest
64         (``video_id`` + rally output + provenance); when a local ``source_path`` is given,
also stores the
65         source video. Returns the written URIs. Reuses :func:`backend.storage.put` + the
``VideoWorkspace``
66         layout, so it honours whatever ``storage.backend`` the deploy uses (local / gcs /
s3)."""
67         written: Dict[str, str] = {}
68         cfg = _sc(storage_config)
69         fd, manifest_local = tempfile.mkstemp(suffix=".json")
70         try:
71             with os.fdopen(fd, "w", encoding="utf-8") as fh:
72                 json.dump(
73                     {"video_id": workspace.video_id, "rally_output": rally_output},
74                     fh,
75                     indent=2,
76                 )
77             manifest_uri = _workspace.join_uri(workspace.outputs, "capture.json")
78             written["manifest_uri"] = _storage.put(manifest_local, manifest_uri, config=cfg)
79             if source_path and os.path.exists(source_path):
80                 source_uri = _workspace.join_uri(
81                     workspace.source, os.path.basename(source_path)
82                 )
83                 written["source_uri"] = _storage.put(source_path, source_uri, config=cfg)
84         finally:
85             try:
86                 os.unlink(manifest_local)
87             except OSError:
88                 pass
89         return written
90
91
92     def shadow_eval(
93         candidates: Sequence[Any],
94         owned_variant: Any,
95         gemini_variant: Any,
96         *,
97         labeled: bool = False,
98         iou: float = 0.5,
99     ) -> Dict[str, Any]:

```

34. user (2026-07-08T05:47:00.222Z)

```

backend\compute\__init__.py:86:     region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
backend\compute\__init__.py:92:         region=region,
backend\compute\cloud_run.py:75:         region: str,
backend\compute\cloud_run.py:113:         region: str,

```

```

backend\compute\cloud_run.py:134:         name = client.job_path(project, region, job_name)
backend\compute\cloud_run.py:222:         region: str,
backend\compute\cloud_run.py:231:         self._region = region
backend\compute\cloud_run.py:272:         self._job_name, argv, region=self._region,
project=self._project
backend\compute\cloud_run.py:355:         f"--region {self._region}`."

```

35. user (2026-07-08T05:47:10.521Z)

```

60         server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
61         """
62         target = (
63             target_override
64             if target_override is not None
65             else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
66         )
67         if target != "cloud_run":
68             return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
69
70         # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
71         from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
72
73         # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
74         # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout
so a
75         # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild §4.3; the
old
76         # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps
an
77         # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
78         if policy is DEFAULT_POLICY:
79             max_wait = getattr(
80                 getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
81             )
82             if max_wait:
83                 policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))
84
85         job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
86         region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
87         project = os.environ.get("GOOGLE_CLOUD_PROJECT")
88         client = run_client if run_client is not None else GoogleCloudRunJobClient()
89         return CloudRunCompute(
90             run_client=client,
91             job_name=job_name,
92             region=region,
93             project=project,
94             storage_config=_storage_dict(config),
95             policy=policy,
96             token=token,
97         )
98

```

36. user (2026-07-08T05:47:11.720Z)

```

1490
1491         # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
1492         if args.video_path and not args.server and not args.app:
1493             print(f"[CLI] Processing video file: {args.video_path}")
1494             # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
1495             # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the

```

```

1496         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
1497         storage_cfg = getattr(global_config, "storage", None)
1498         try:
1499             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1500         except ValueError as e:
1501             print(f"[FAIL] {e}")
1502             return
1503         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1504             print(f"[FAIL] Video file not found: {args.video_path}")
1505             return
1506         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1507         try:
1508             video_id = compute_video_id(local_video)
1509             was_reingest = db_instance.get_video(video_id) is not None
1510
1511             # Validate (with-context variant surfaces ffprobe_meta for the report)
1512             print("[CLI] Validating...")
1513             val_results, val_context = registry.run_all_with_context(
1514                 local_video, global_config
1515             )
1516             failures = [r for r in val_results if not r.passed]
1517
1518             # Save video status
1519             status = "failed" if failures else "processed"
1520             db_instance.add_video(
1521                 video_id, args.video_path, status, [r.model_dump() for r in val_results]
1522             )
1523
1524             seg_name = args.segmenter or getattr(
1525                 getattr(global_config, "indexing", None), "default_segmenter", "motion"
1526             )
1527             segmenter_actual = None
1528             segmentation_ran = False
1529             try:
1530                 print("\n" + "=" * 40)
1531                 print("VALIDATION SUMMARY")
1532                 print("=" * 40)
1533                 for r in val_results:
1534                     status_symbol = "[PASS]" if r.passed else "[FAIL]"
1535                     print(f"{status_symbol} {r.validator_name}: {r.message}")
1536                 print("=" * 40 + "\n")
1537
1538                 if failures:
1539                     print("[FAIL] Video failed checks. Indexing halted.")
1540                     return
1541
1542                 # Index
1543                 segmenter_cls = segmenter_registry.get(seg_name)
1544                 if not segmenter_cls:
1545                     print(f"[FAIL] Segmenter '{seg_name}' not found.")
1546                     return
1547
1548                 print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1549                 segmenter_actual = seg_name
1550                 play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1551                 segmenter = segmenter_cls(db_instance, global_config)
1552                 result = segmenter.process_video(
1553                     video_id, local_video, max_frames=args.max_frames
1554                 )
1555                 segmentation_ran = True
1556
1557                 if isinstance(result, Failure):
1558                     print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1559                     print("[FAIL] Indexing halted for video.")

```

```

1560         segments = []
1561     else:
1562         segments = result.value if hasattr(result, "value") else result
1563         print(
1564             f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}"
1565         )
1566         # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1567         _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1568     finally:
1569         # Always emit the per-video observability report. Never raises.
1570         paths = emit_indexer_report(
1571             db=db_instance,
1572             video_id=video_id,
1573             # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI
would write a
1574             # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source
URI is kept
1575             # in DB provenance via db.add_video.
1576             video_path=local_video,
1577             config=_report_config_for_input(global_config, args.video_path),
1578             val_results=val_results,
1579             val_context=val_context,
1580             segmenter_requested=args.segmenter,
1581             segmenter_actual=segmenter_actual,
1582             was_reingest=was_reingest,
1583             segmentation_ran=segmentation_ran,
1584         )
1585         if paths:
1586             print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1587             print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1588             print(f"[REPORT] Machine JSON      : {paths.get('json')}")
1589         finally:
1590             storage.cleanup_acquired_local_input(
1591                 args.video_path, local_video, storage_cfg
1592             )
1593         return
1594
1595     # Start Local Server Mode
1596     if args.app or args.server or not args.video_path:
1597         if args.app:
1598             # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1599             # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.

```

37. user (2026-07-08T05:47:23.658Z)

```

backend\api\uploads.py:185:         "hasher": hashlib.md5(),
backend\utils\hashing.py:29:     hasher = hashlib.md5()

```

38. user (2026-07-08T05:47:25.672Z)

```

deploy\cloudrun\Dockerfile:3:# Runs `python -m backend.worker infer ?` INLINE on an L4 (detect +
window + stitch co-located so
deploy\cloudrun\README.md:13:- GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region
(asia-south1, else Singapore ? see
deploy\cloudrun\README.md:20:# L4 GPU is region-limited ? use asia-southeast1 (Singapore), NOT
asia-south1 (Mumbai).
deploy\gcp\teardown.sh:26:ZONE="${RALLY_GCP_ZONE:-asia-south1-a}"
docs\COMPUTE_DECOUPLED_SERVING\README.md:31:| D4 | In **cloud-serving, STITCH runs on Cloud
Run** (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger. |
owner (a) | Stitch-on-laptop would hide real cost. |
docs\COMPUTE_DECOUPLED_SERVING\README.md:43:[Omitted long matching line]

```

```

docs\COMPUTE_DECOUPLED_SERVING\README.md:54:| **ADR-010** | DECOUPLED: DO?GCP pivot | The cloud-
compute origin (GPU + co-located bucket). |
docs\COMPUTE_DECOUPLED_SERVING\README.md:91:[Omitted long matching line]
docs\COMPUTE_DECOUPLED_SERVING\README.md:140:2. ? **Stitch placement ? co-located** in the
detect job (= D4; one metered unit; the job table supports a future CPU-split non-breaking,
revisit only on bursty-compile metrics).
deploy\gcp\README.md:1:# GCP provisioning runbook ? DECOUPLED_COMPUTE GPU + storage (asia-
south1)
deploy\gcp\README.md:7:> runbook reuses. ? Capacity note: asia-south1 (Mumbai) GPUs are
frequently stocked-out ? sweep zones /
deploy\gcp\README.md:8:> fall back to asia-southeast1 (Singapore) etc.; weights still land in
the asia-south1 bucket.
deploy\gcp\README.md:10:Co-located **GPU VM + GCS bucket in ONE region** (asia-south1 / Mumbai)
for the first real
deploy\gcp\README.md:13:has GPUs in asia-south1 (Mumbai) and asia-southeast1 (Singapore). Owner
picked **Mumbai** (closest
deploy\gcp\README.md:34:| GCS bucket | `gs://khehsutra-rally-corpus` (asia-south1, UBLA) |
~$0.02/GB/mo storage | `gcloud storage rm -r` (? data loss) |
deploy\gcp\README.md:65:asia-south1 NVIDIA_T4_GPUS = 1 NVIDIA_L4_GPUS = 1 (regional: OK)
deploy\gcp\README.md:106: --zone=asia-south1-a \
deploy\gcp\README.md:124:> on it. **T4 was capacity-stocked-out in `asia-south1-a`; an L4
(`g2-standard-4`, no `--accelerator`
deploy\gcp\README.md:125:> flag ? L4 is built into g2) in `asia-south1-c` landed** ? try
L4/other zones if T4 is unavailable.
deploy\gcp\README.md:150:> If `asia-south1-a` reports no T4 capacity, try `-b` / `-c`. The
Ubuntu image needs the NVIDIA
deploy\gcp\README.md:156:SSH: `gcloud compute ssh rally-gpu --zone=asia-south1-a`.
deploy\gcp\README.md:172:gcloud compute scp /tmp/rally.tgz rally-gpu:/root/ --zone=asia-south1-a
deploy\gcp\README.md:173:gcloud compute ssh rally-gpu --zone=asia-south1-a -- 'mkdir -p ~/rally
&& tar -xzf ~/rally.tgz -C ~/rally'
deploy\gcp\README.md:185:gcloud compute scp ~/.gcp/rally-worker-sa.json rally-
gpu:/root/.gcp/sa.json --zone=asia-south1-a
deploy\gcp\README.md:210:# stage the 7 Done videos + 7 labels into the bucket (from the co-
located box ? fast):
deploy\gcp\README.md:227:- **GCS bucket (recommended, co-located):** `gcs://khehsutra-rally-
corpus/videos/clip.mp4`. Upload with
deploy\gcp\README.md:256:# cross-region VM: set the zone (teardown defaults to asia-south1-a)
deploy\gcp\provision.sh:2:# Provision the DECOUPLED_COMPUTE GCP storage + IAM (asia-south1 /
Mumbai). Idempotent: safe to
deploy\gcp\provision.sh:15:REGION="${RALLY_GCP_REGION:-asia-south1}"
deploy\gcp\provision.sh:33:echo "== GCS bucket (asia-south1, uniform bucket-level access) =="
deploy\gcp\create_gpu_vm.sh:7:# It sweeps zones for GPU capacity ? asia-south1 (Mumbai, co-
located with the bucket) first, then
deploy\gcp\create_gpu_vm.sh:31:ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-
south1-c asia-southeast1-a asia-southeast1-b asia-southeast1-c}"
deploy\gcp\nightly_regression\create_nightly_bucket.sh:21:LOCATION="${RALLY_GCP_LOCATION:-asia-
south1}"
deploy\gcp\nightly_regression\launch.sh:40:ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b
asia-south1-c asia-southeast1-a asia-southeast1-b asia-southeast1-c}"
deploy\gcp\nightly_regression\README.md:38:shown). `J=nightly-regression-launcher`, `S=nightly-
regression`, `R=asia-south1`, `P=gen-lang-client-0306026826`.
deploy\gcp\nightly_regression\README.md:98:gcloud run jobs execute nightly-regression-launcher
--region asia-south1 --project gen-lang-client-0306026826
deploy\gcp\nightly_regression\README.md:111:- **Launcher logs**: `gcloud run jobs executions
list --job=nightly-regression-launcher --region=asia-south1` then `? executions logs read <exec>
--region=asia-south1`.
deploy\gcp\nightly_regression\README.md:117:gcloud scheduler jobs pause nightly-regression
--location asia-south1 # stop nightly firing (keep everything)
deploy\gcp\nightly_regression\README.md:118:gcloud scheduler jobs resume nightly-regression
--location asia-south1 # turn it back on
deploy\gcp\nightly_regression\README.md:119:# mute email but keep running: gcloud run jobs
update nightly-regression-launcher --region asia-south1 --update-env-vars RALLY_EMAIL=false

```

[Showing results with pagination = limit: 40]

39. user (2026-07-08T05:47:39.584Z)

```
backend\main.py:258:class ProcessRequest(BaseModel):
backend\main.py-259-     video_path: str
backend\main.py-260-     player_pool: Optional[List[str]] = None
backend\main.py-261-     max_frames: Optional[int] = None
backend\main.py-262-     segmenter: Optional[str] = None
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-263- # The
UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job for PMF
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-264- #
analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older clients).
backend\main.py-265-     locale: Optional[str] = None
backend\main.py-266-     # SPA compute-picker (item 3): "local" forces in-process even on a
cloud-configured server;
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-267- #
"cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ? server
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-268- #
default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
backend\main.py-269-     compute: Optional[str] = None
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-270- #
Explicit opt-in reprocess path. By default, a video whose content hash already has stored
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-271- #
play_segments is returned from cache so refresh/Try again cannot re-bill paid segmenters.
backend\main.py-272-     force: bool = False
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-273-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-274-
backend\main.py-275:class AnnotationsRequest(BaseModel):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-276- #
Finished rally labels for a video, as the annotator's byte-compatible CSV (P8, from the Studio
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-277- #
AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is written verbatim.
backend\main.py-278-     video_id: str
backend\main.py-279-     csv: str
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-280-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py-281-
backend\main.py-282:class AssetRequest(BaseModel):
backend\main.py-283-     # "Store this video in my medium" (STUDIO_MEDIA_LIFECYCLE.md P4, D14).
Exactly ONE source:
```

40. assistant (2026-07-08T05:48:57.496Z)

I have the full picture now. Let me report the structured map.

41. user (2026-07-08T05:51:07.738Z)

Structured output provided successfully